

Chapter 4

組合邏輯

組合電路

組合電路 (**Combinational circuit**)

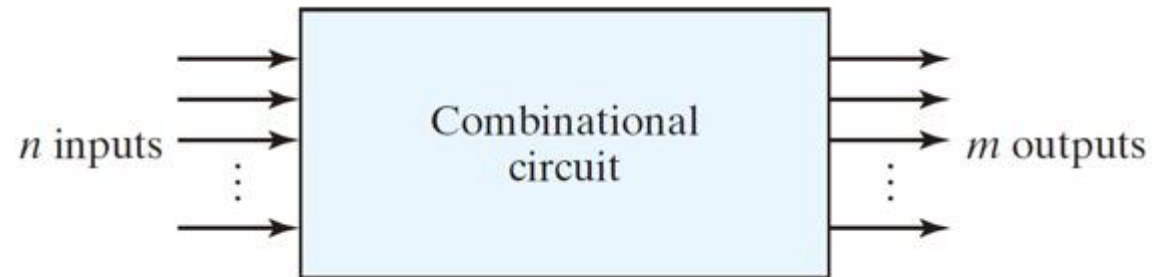
由邏輯閘所組成，其在任何時刻的輸出僅由當前的輸入組合決定。

循序電路 (**Sequential circuit**)

包含記憶元件，
其輸出是目前輸入與記憶元件狀態的函數，
同時輸出也會受到過去輸入的影響。

組合電路

- ▶ 對於 n 個輸入變數而言，將有 2^n 種可能的二進位輸入組合

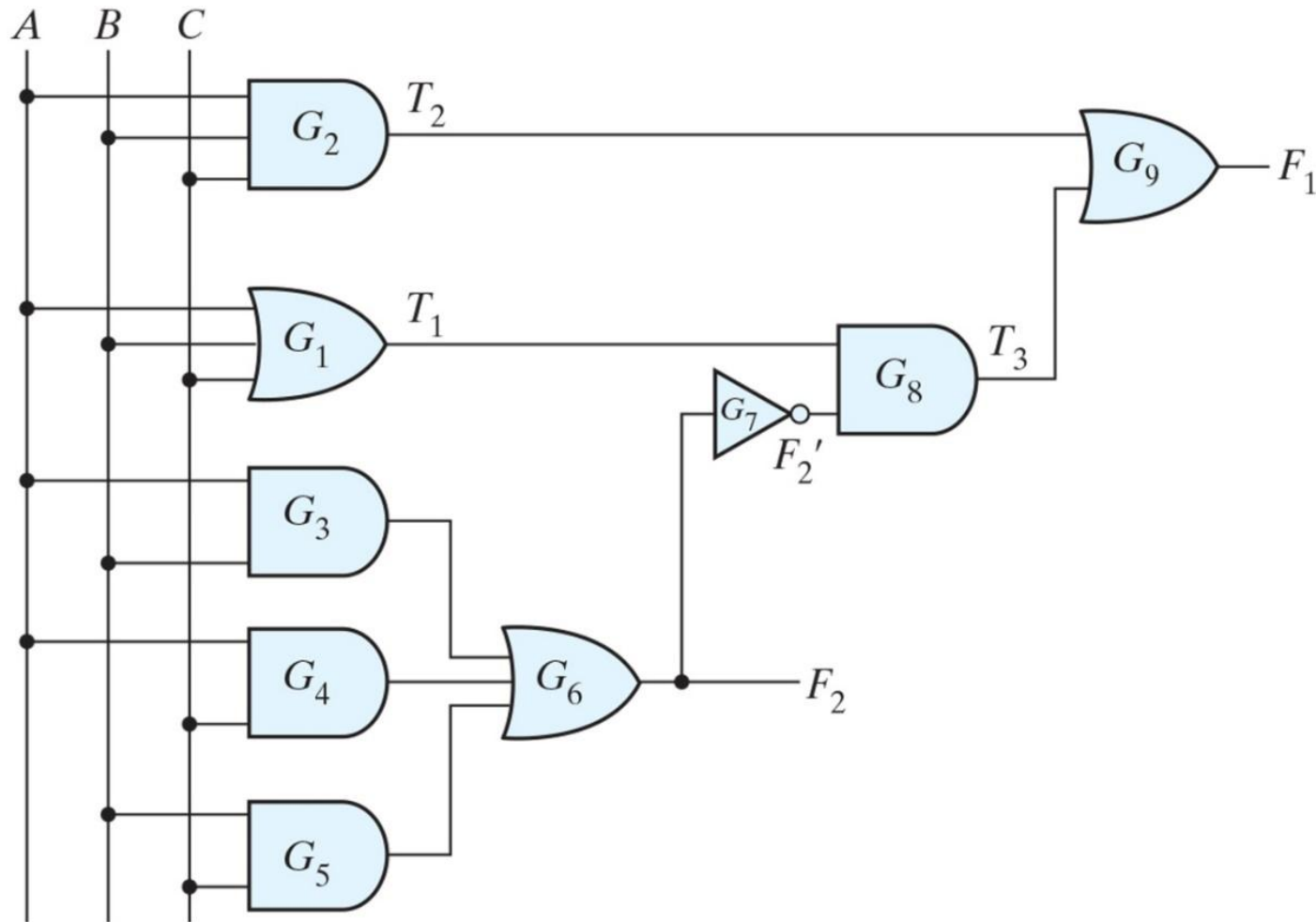


- ▶ 標準組合電路
 - ▶ 加法器(adders)、減法器(subtractors)、比較器(comparators)、解碼器(decoders)、編碼器(encoders)，及多工器(multiplexers)。

組合電路的分析

- ▶ 組合電路的分析就是需要去決定該電路所執行的函數。
 - ▶ 確認該電路是組合電路而非循序電路
 - ▶ 沒有回授路徑 (feedback path)
 - ▶ 沒有記憶元件 (memory elements)
 - ▶ 推導其布林函數 (或真值表)
 - ▶ 設計驗證 (design verification)
 - ▶ 提供其功能的文字說明 (verbal explanation)

組合電路的分析



$$\begin{aligned}F_2 &= AB+AC+BC \\T_1 &= A+B+C \\T_2 &= ABC \\T_3 &= F_2'T_1 \\F_1 &= T_3+T_2\end{aligned}$$

Figure 4.2 Logic diagram for analysis example.

組合電路的分析

► $F_1 = T_3 + T_2 = F_2' T_1 + ABC$
 $= (AB + AC + BC)'(A + B + C) + ABC$
 $= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC$
 $= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC$
 $= A'BC' + A'B'C + AB'C' + ABC$

- A full-adder
- F_1 : the sum
 - F_2 : the carry

Table 4.1
Truth Table for the Logic Diagram of Fig. 4.2

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i> ₂	<i>F</i> ₂ '	<i>T</i> ₁	<i>T</i> ₂	<i>T</i> ₃	<i>F</i> ₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

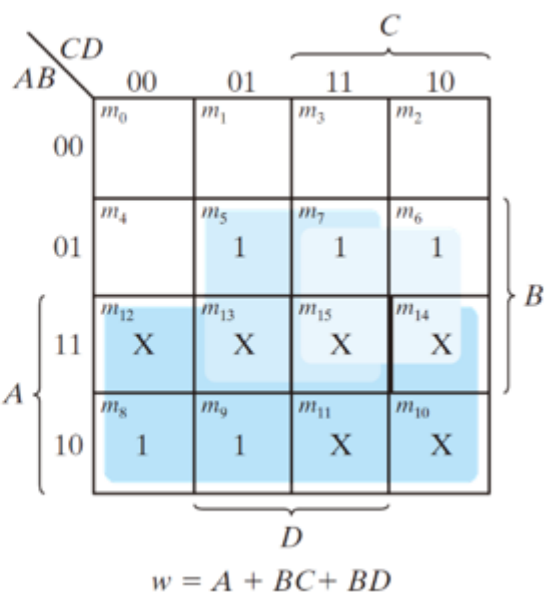
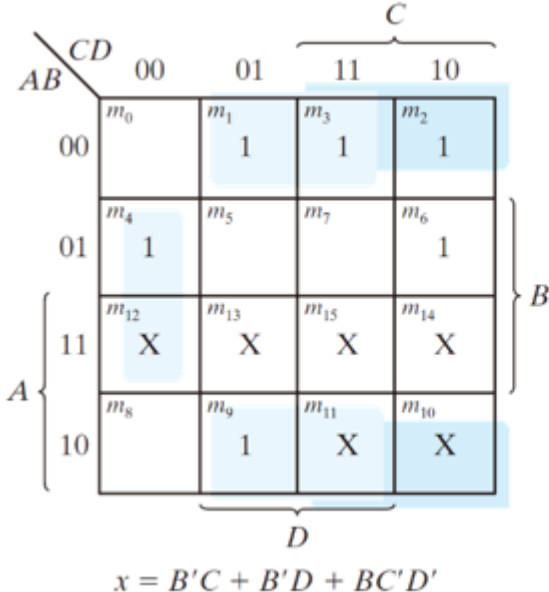
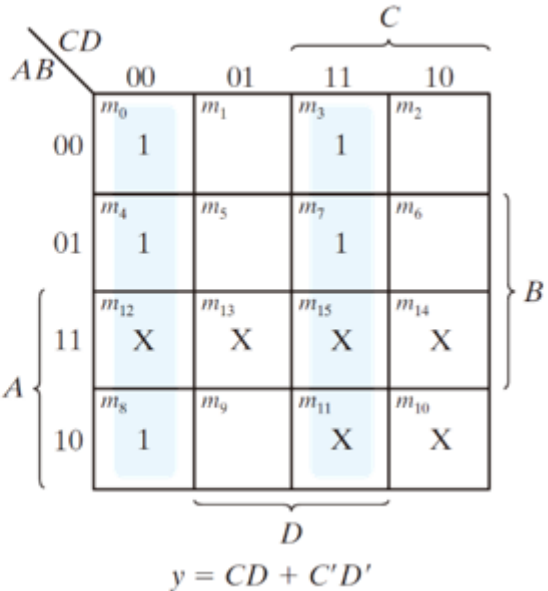
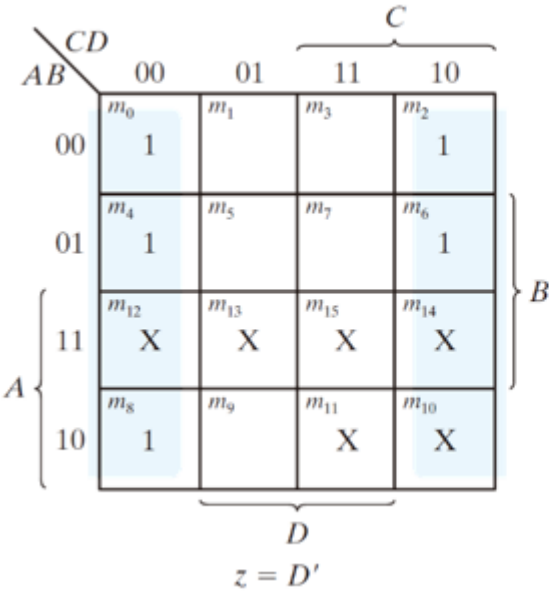
設計步驟

- ▶ 組合電路的設計流程 (The design procedure of combinational circuits)
 1. 陳述問題 (系統規格) (State the problem, system specification)
 - ▶ 決定輸入與輸出 (determine the inputs and outputs)
 - ▶ 為輸入與輸出變數指定符號 (the input and output variables are assigned symbols)
 2. 建立真值表 (derive the truth table)
 3. 推導並化簡布林函數 (derive the simplified Boolean functions)
 4. 繪製邏輯電路圖並驗證正確性 (draw the logic diagram and verify the correctness)

BCD 碼轉換成超3碼的例子

表 4.2 碼轉換例子的真值表

BCD 輸入				超 3 碼輸出			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0



BCD 碼轉換成超3碼的例子

▶ 簡化函數

▶ $z = D'$

▶ $y = CD + C'D'$

▶ $x = B'C + B'D + BC'D'$

▶ $w = A + BC + BD$

▶ 另一種實現方式

▶ $z = D'$

▶ $y = CD + C'D' = CD + (C+D)'$

▶ $x = B'C + B'D + BC'D' = B'(C+D) + B(C+D)'$

▶ $w = A + BC + BD$

Logic Diagram

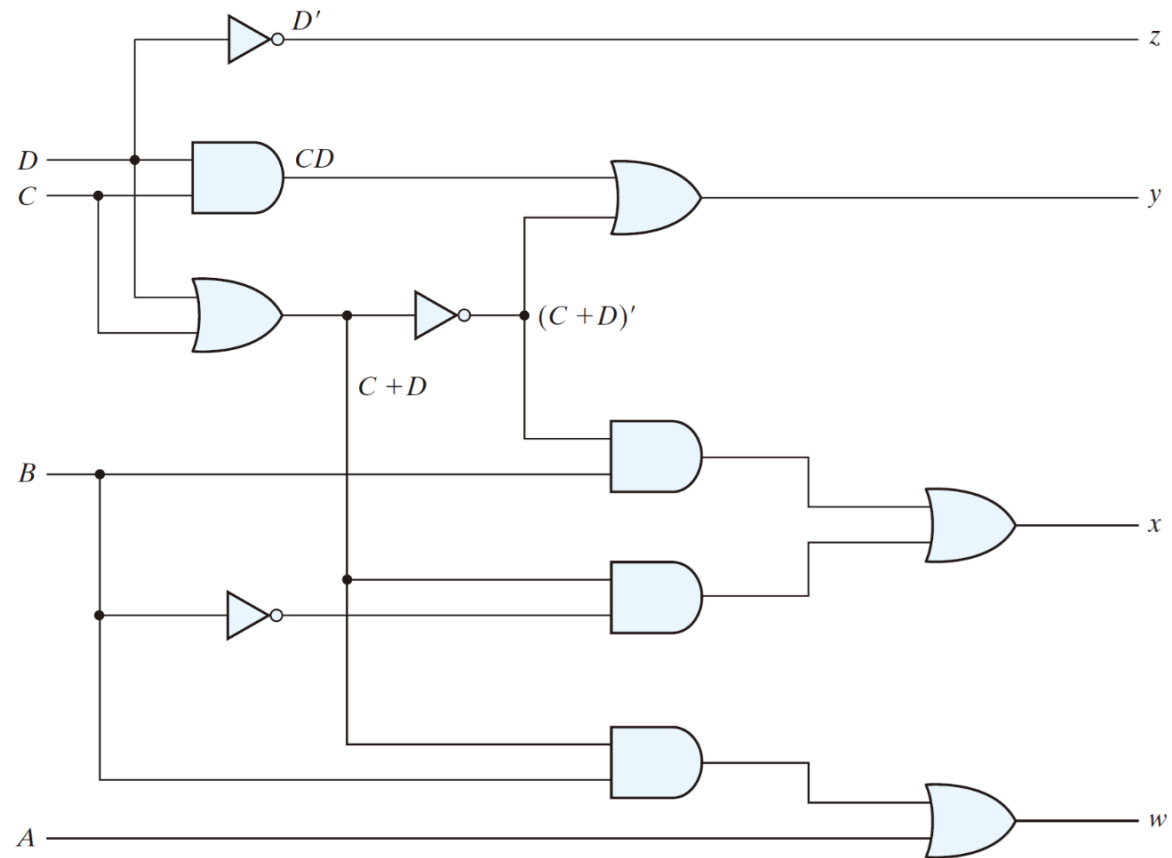


FIGURE 4.4
Logic diagram for BCD-to-excess-3 code converter

二進位加法器

二進位加法器-減法器

► 加法

- $0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1, 1 + 1 = 10$
- 但當被加數與加數都等於1 時，兩者的和就變成為二位數。所得結果中較高的有效位元，稱為**進位(carry)**
- 實現兩個位元相加的組合電路，稱為**半加法器(half adder)**
- 執行三個位元(兩個有效位元和一個先前進位) 相加的電路，即為一個**全加法器(full adder)**

半加法器(half adder)

- ▶ Two input

 - ▶ x, y

- ▶ Two output

 - ▶ $C(\text{carry}), S(\text{sum})$

Table 4.3
Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

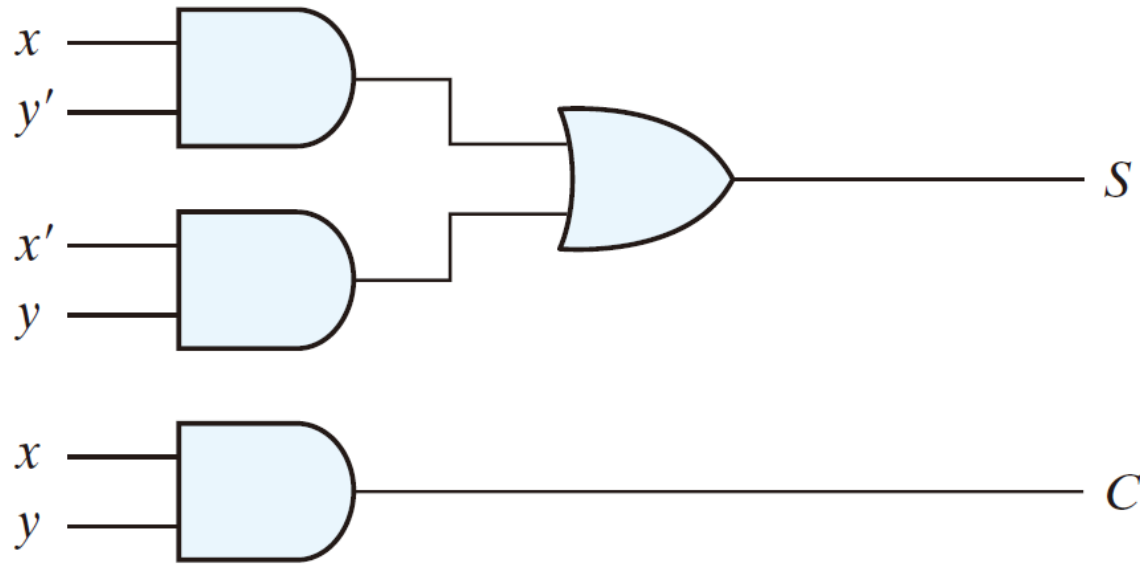
半加法器(half adder)

- ▶ $S = x'y + xy'$
- ▶ $C = xy$
- ▶ the flexibility for implementation
- ▶ $S = x \oplus y$
- ▶ $S = (x+y)(x'+y')$
- ▶ $S' = xy + x'y'$
 $S = (C + x'y')'$
- ▶ $C = xy = (x' + y')'$

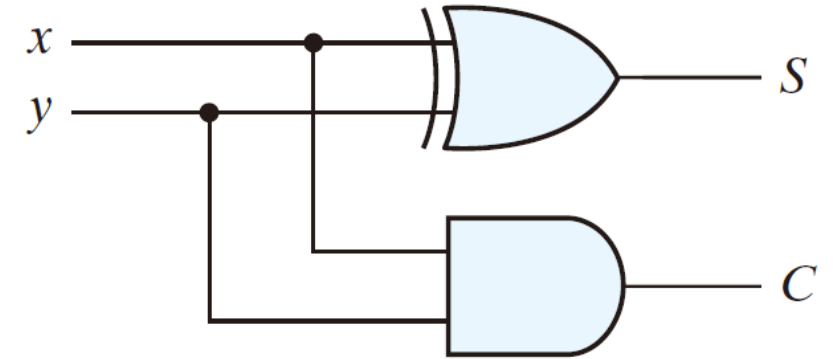
Table 4.3
Half Adder

<i>x</i>	<i>y</i>	<i>c</i>	<i>s</i>
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

半加法器(half adder)



$$\begin{aligned} \text{(a)} \quad S &= xy' + x'y \\ C &= xy \end{aligned}$$



$$\begin{aligned} \text{(b)} \quad S &= x \oplus y \\ C &= xy \end{aligned}$$

FIGURE 4.5

Implementation of half adder

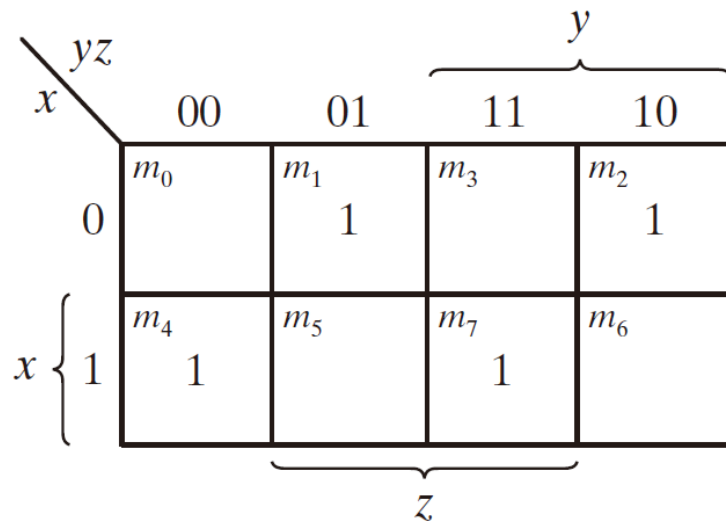
全加法器(full-adder)

- ▶ The arithmetic sum of three input bits
- ▶ **Three input bits**
 - ▶ x, y: two significant bits
 - ▶ z: the carry bit from the previous lower significant bit
- ▶ **Two output bits: C, S**

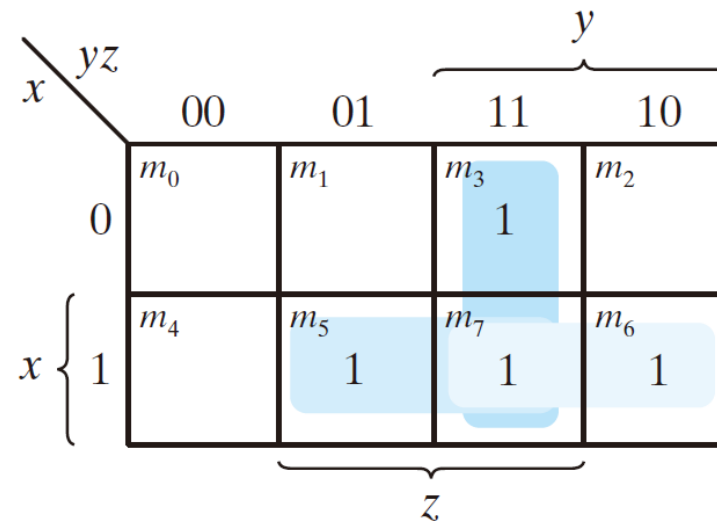
Table 4.4
Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

全加法器(full-adder) - 卡諾圖



(a) $S = x'y'z + x'yz' + xy'z' + xyz$



(b) $C = xy + xz + yz$

FIGURE 4.6
K-Maps for full adder

全加法器(full-adder)

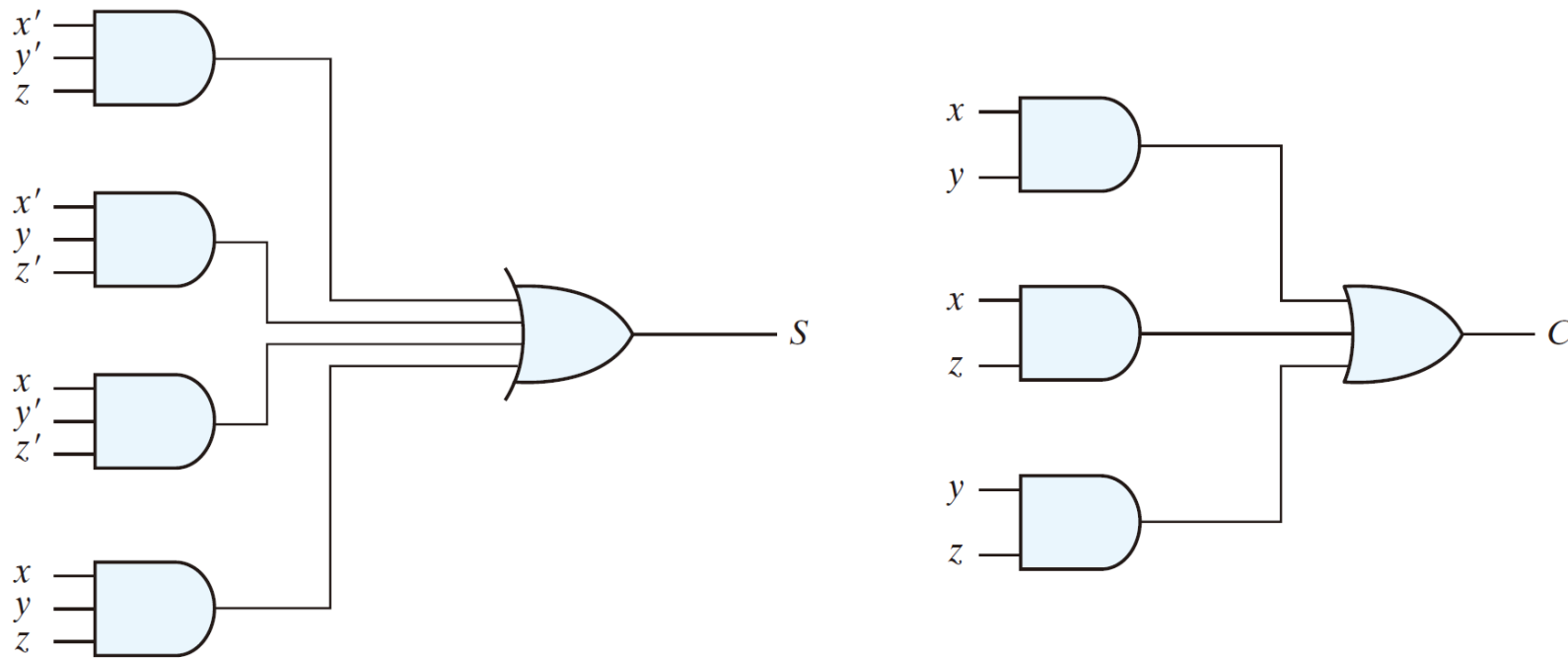


FIGURE 4.7
Implementation of full adder in sum-of-products form

► $S = x'y'z + x'yz' + xy'z' + xyz$

$C = xy + xz + yz$

► $S = z \oplus (x \oplus y)$

$= z' (xy' + x'y) + z(xy' + x'y)$

$= z'xy' + z'x'y + z((x' + y)(x + y'))$

$= xy'z' + x'yz' + xyz + x'y'z$

$C = z(xy' + x'y) + xy$

$= xy'z + x'yz + xy$

$= z(x \oplus y) + xy$

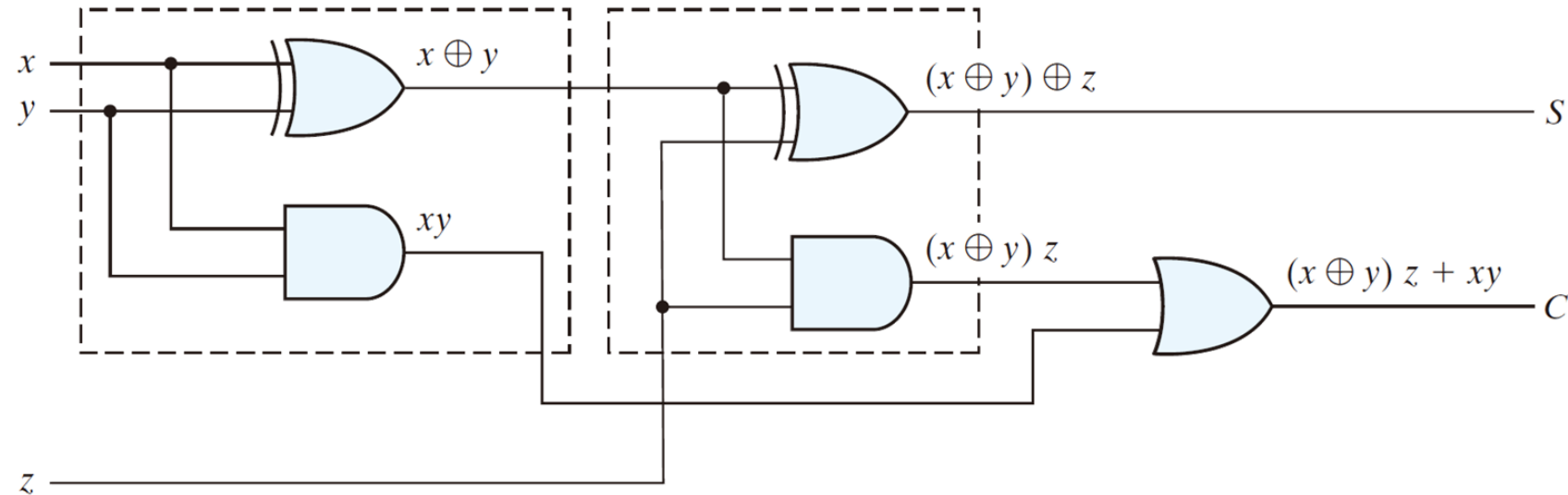


FIGURE 4.8

Implementation of full adder with two half adders and an OR gate

二進位加法器

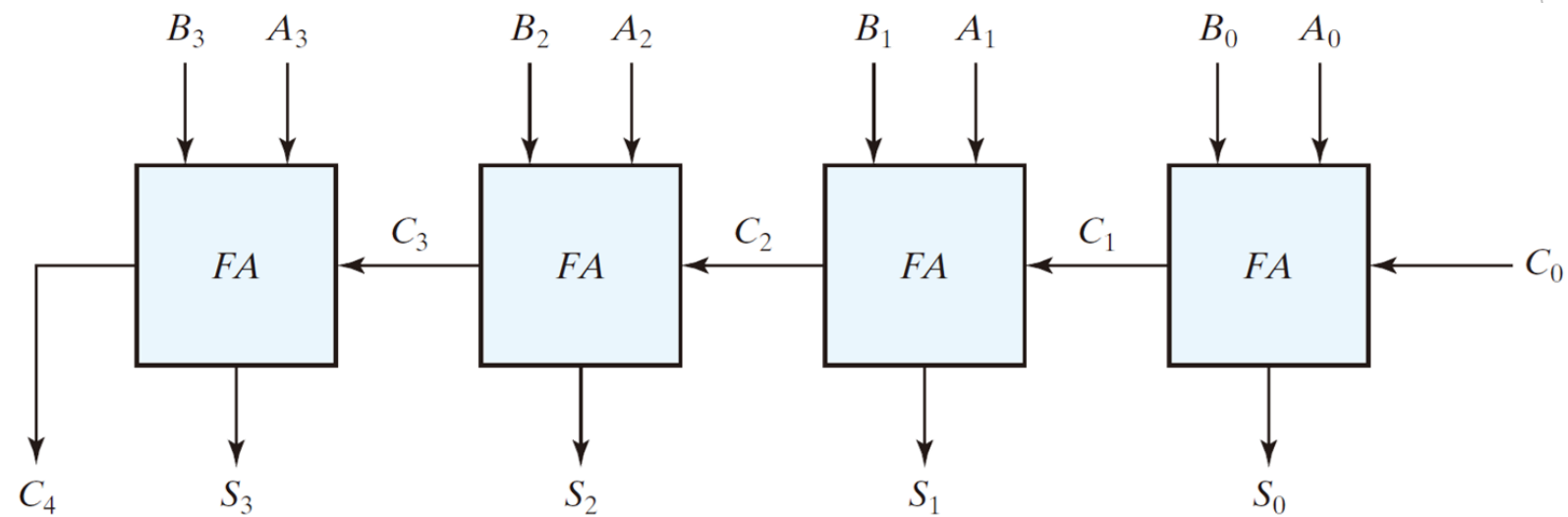


FIGURE 4.9
Four-bit adder

漣波進位加法器

Subscript <i>i</i> :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

進位傳播

- ▶ 在正確的輸出總和在輸出端出現以前，信號必須經過閘的傳播。
 - ▶ 在正確的輸出總和和輸出端出現以前，信號必須經過閘的傳播。
 - ▶ 電路不是「瞬間算完」
- ▶ 訊號會一層一層經過邏輯閘（gate delay）
- ▶ 加法器中的最長傳播延遲時間是使得進位傳遞過全加法器所要花的時間。

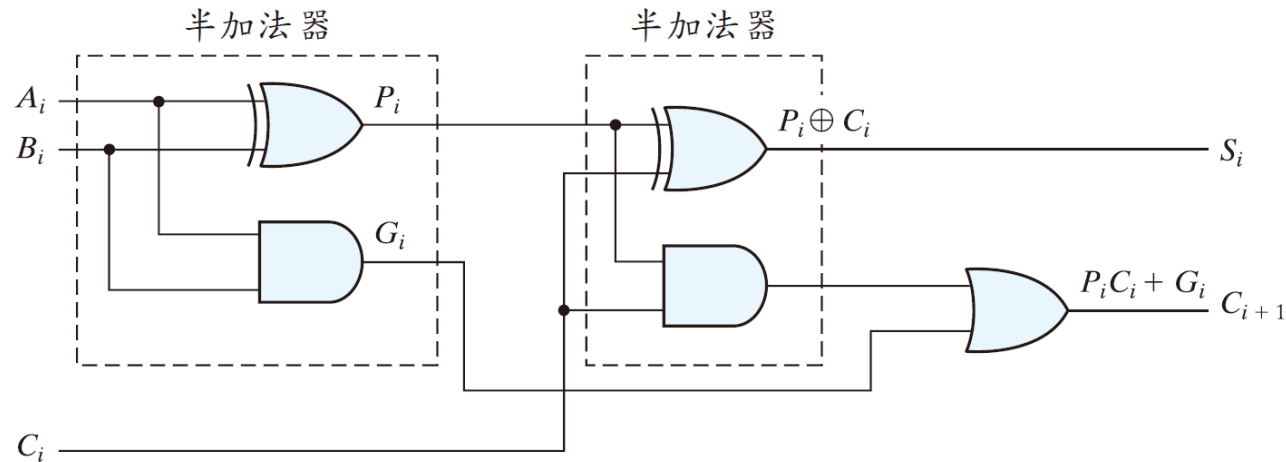


圖 4.10 具有顯示 P 和 G 的全加法器

減少進位傳播延遲時間

- ▶ 使用具有減少延遲的快速閘
- ▶ 應用**前瞻進位邏輯 (CLA)**

- ▶ carry propagate: $P_i = A_i \oplus B_i$

- ▶ carry generate: $G_i = A_i B_i$

- ▶ sum: $S_i = P_i \oplus C_i$

- ▶ carry: $C_{i+1} = G_i + P_i C_i$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 (G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 + P_3 P_2 P_1 P_0 C_0$$

邏輯圖

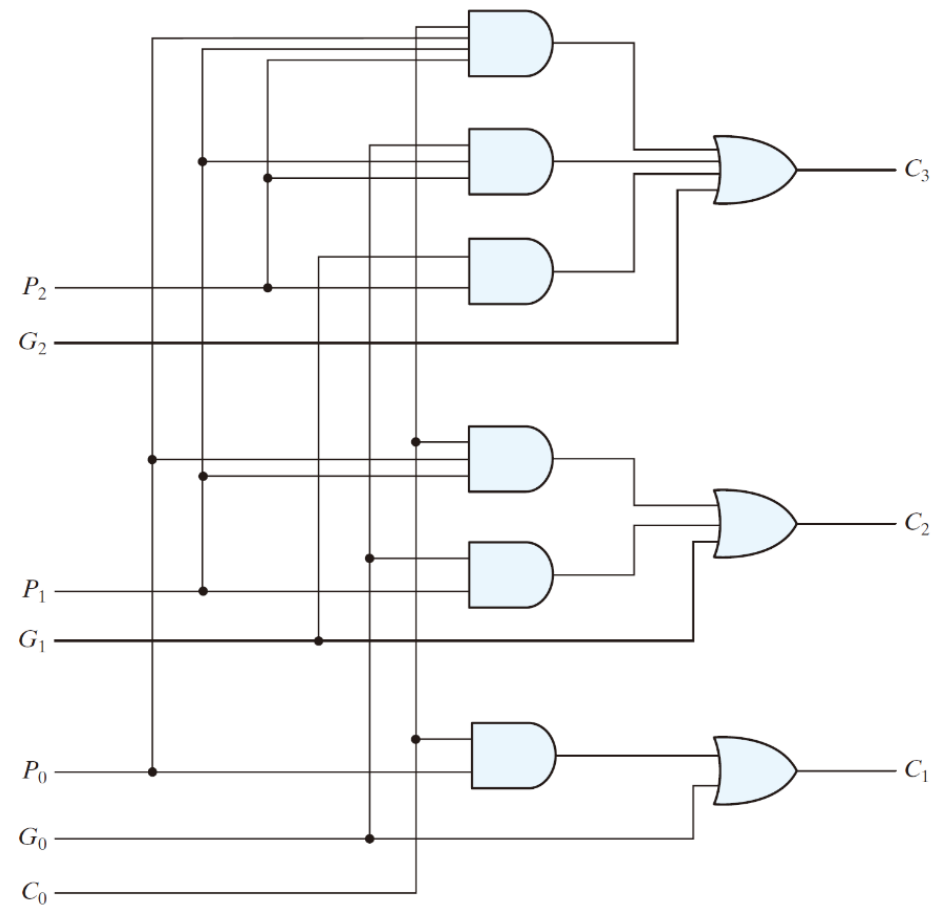


FIGURE 4.11
Logic diagram of carry lookahead generator

二進位減法器

- ▶ $A - B = A + (B \text{ 取2 的補數})$
- ▶ 四位元加法器-減法器電路
 - ▶ 當 $M = 0$ 時，可得 $B \oplus 0 = B$
 - ▶ 當 $M = 1$ 時，可得 $B \oplus 1 = B'$

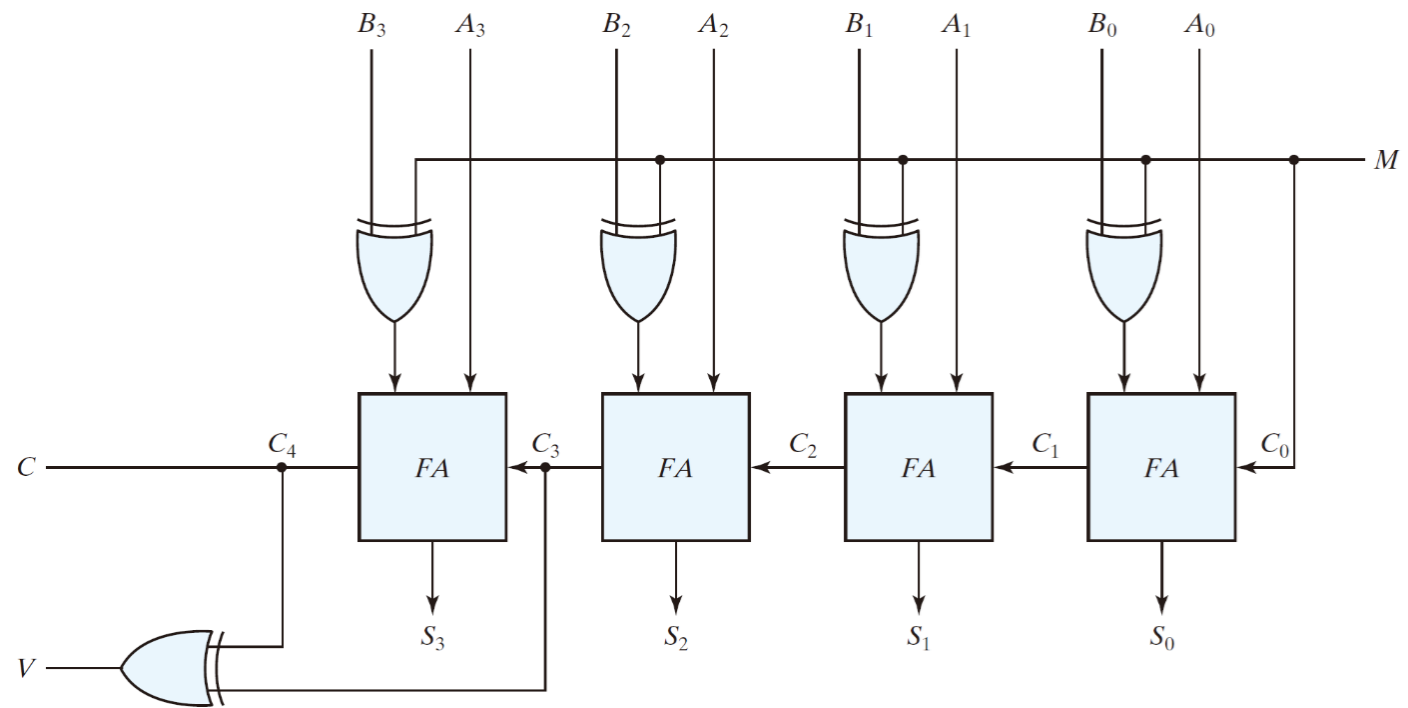


FIGURE 4.13

Four-bit adder-subtractor (with overflow detection)

溢位(Overflow)

- ▶ 容納數字的位元數是有限的
- ▶ 兩個正數相加得到負數
- ▶ 兩個負數相加得到正數
- ▶ $V = 0$, no overflow; $V = 1$, overflow

Example:

carries:

+70
+80

+150

0 1

0 1000110
0 1010000

1 0010110

carries:

-70
-80

-150

1 0

1 0111010
1 0110000

0 1101010

十進位加法器

- ▶ **十進數加法器(decimal adder)** 中，最少需要九個輸入和五個輸出，因為每一個十進數數元需要用四個位元來編碼，並且電路必須有一個輸入及輸出進位。
- ▶ 十進數加法器電路有很多型式，視用來表示十進數數元的碼而定

BCD 加法器

► 將兩個 BCD 數相加

- 9 個輸入：兩個 BCD 數，以及一個進位輸入 carry-in
- 5 個輸出：一個 BCD 數，以及一個進位輸出 carry-out

► 設計方法：

- 使用真值表，需要 2^9 筆輸入組合
- 使用二進位全加器，因為最大總和為：
- $9 + 9 + 1 = 19$

► 也就是說，兩個最大 BCD 數 9 和 9 相加，再加上一個 carry-in，最大結果是 19。

► 將二進位結果轉換成 BCD 表示法

BCD 加法器可以用二進位全加器先完成加法，再將結果轉換或修正為合法的 BCD 輸出。

BCD加法器：真值表

表 4.5 BCD 加法器的推導

$$C = K + (Z_8Z_4 + Z_8Z_2)$$

	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

二進位和					BCD 和					十進位
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19
1	0	0	1	1	1	1	1	0	0	20
1	0	0	1	1	1	1	1	0	1	21
1	0	0	1	1	1	1	1	0	1	22
1	0	0	1	1	1	1	1	0	1	23
1	0	0	1	1	1	1	1	0	1	24
1	0	0	1	1	1	1	1	0	1	25
1	0	0	1	1	1	1	1	0	1	26
1	0	0	1	1	1	1	1	0	1	27
1	0	0	1	1	1	1	1	0	1	28

BCD 加法器

▶ 若和 > 9 ，則需進行修正

▶ $C = 1$

▶ $K = 1$

▶ $Z_8 Z_4 = 1$

▶ $Z_8 Z_2 = 1$

▶ 修正方式： $-(10)_d$ 或 $+6$



$$C = K + Z_8 Z_4 + Z_8 Z_2$$

BCD 加法器

► 方塊圖

$$C = K + Z_8Z_4 + Z_8Z_2$$

輸出
進位

$$+6_{10}$$

$$=0110_2$$

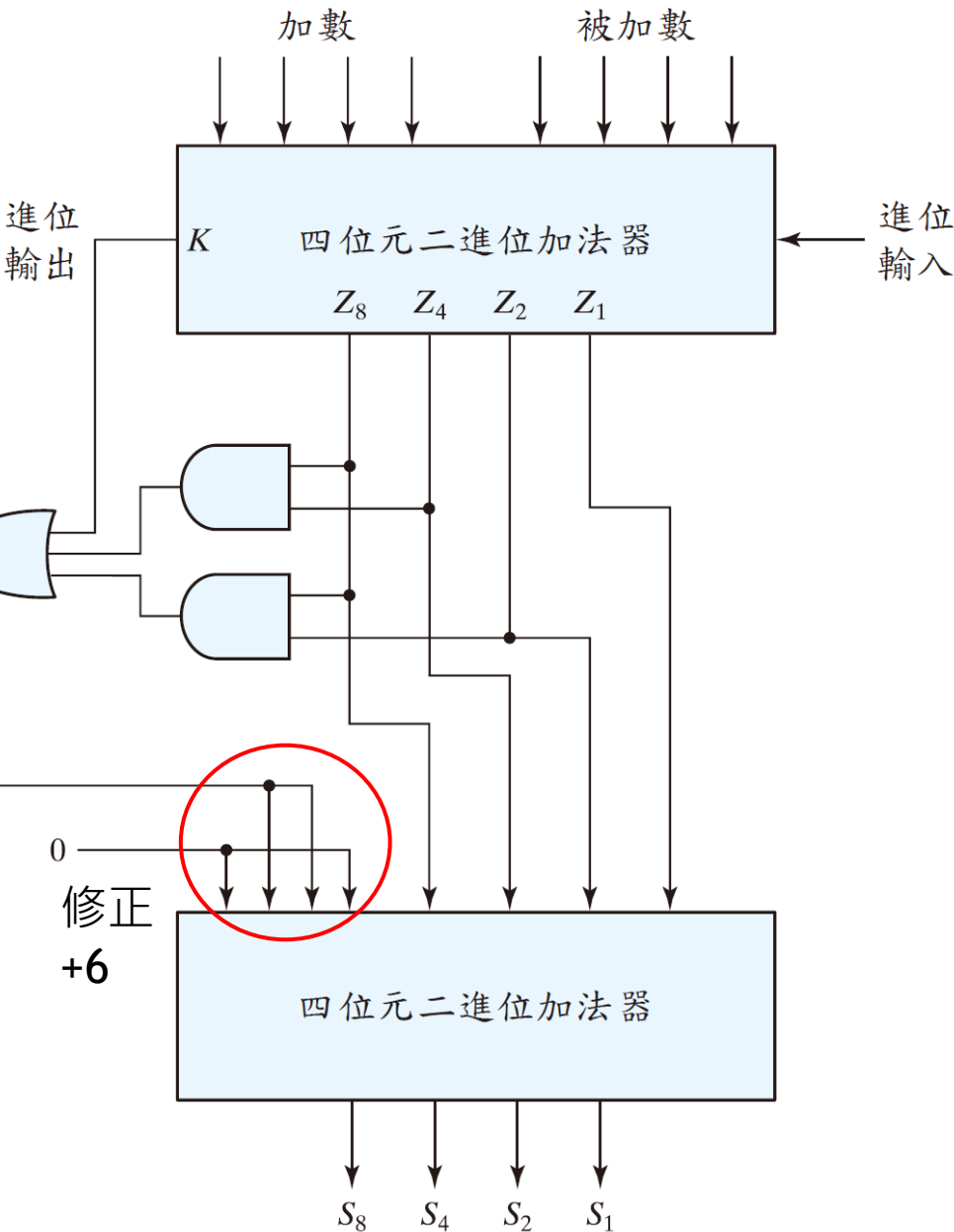
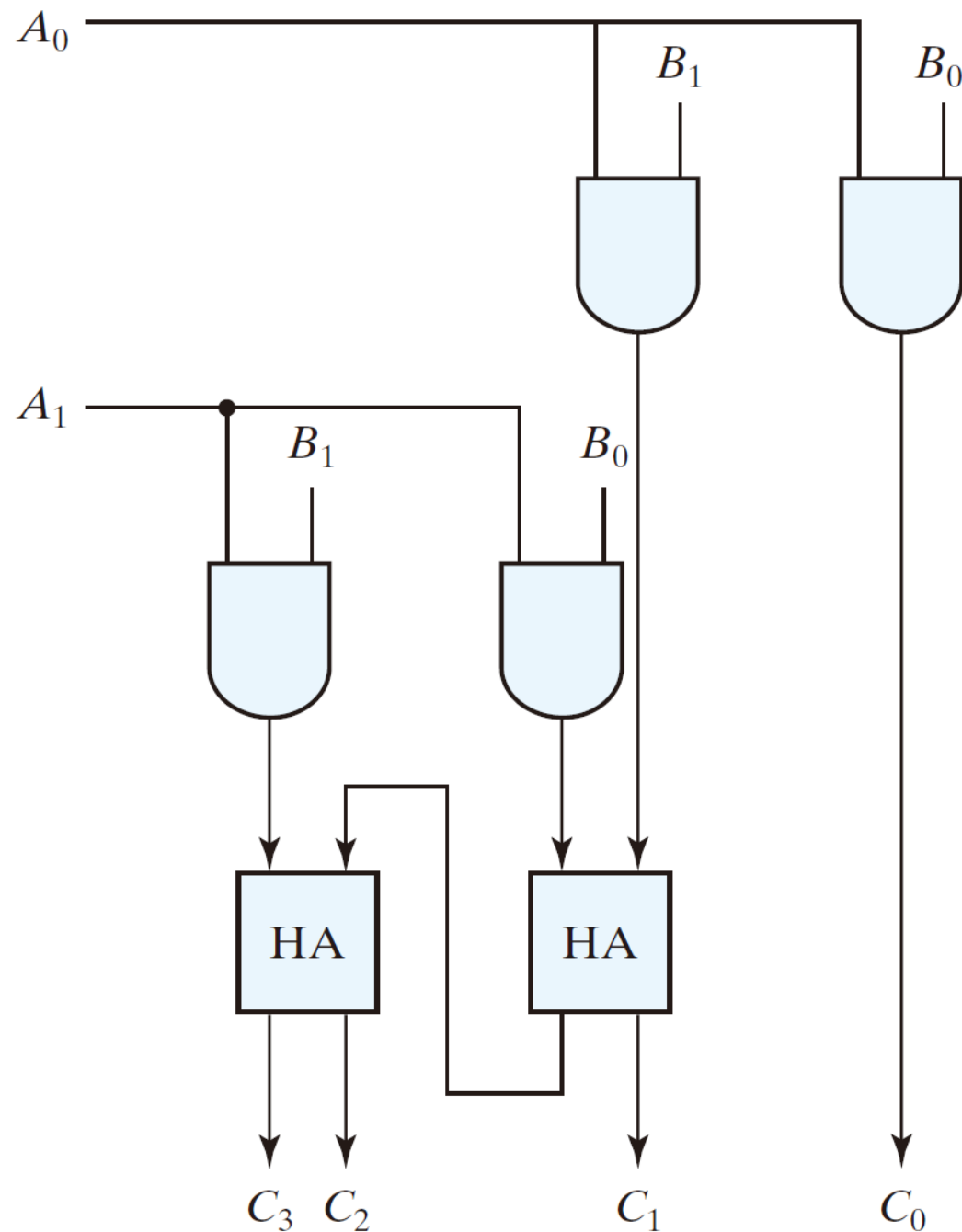
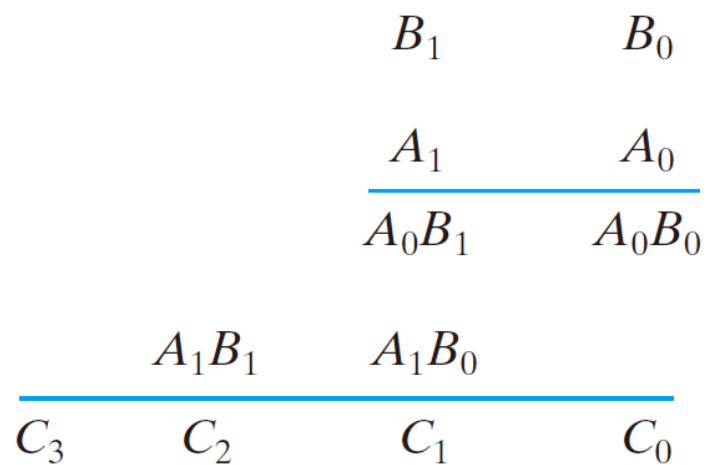


圖 4.14 BCD 加法器的方塊圖

二進位乘法器

- ▶ 部分乘積
- AND 運算



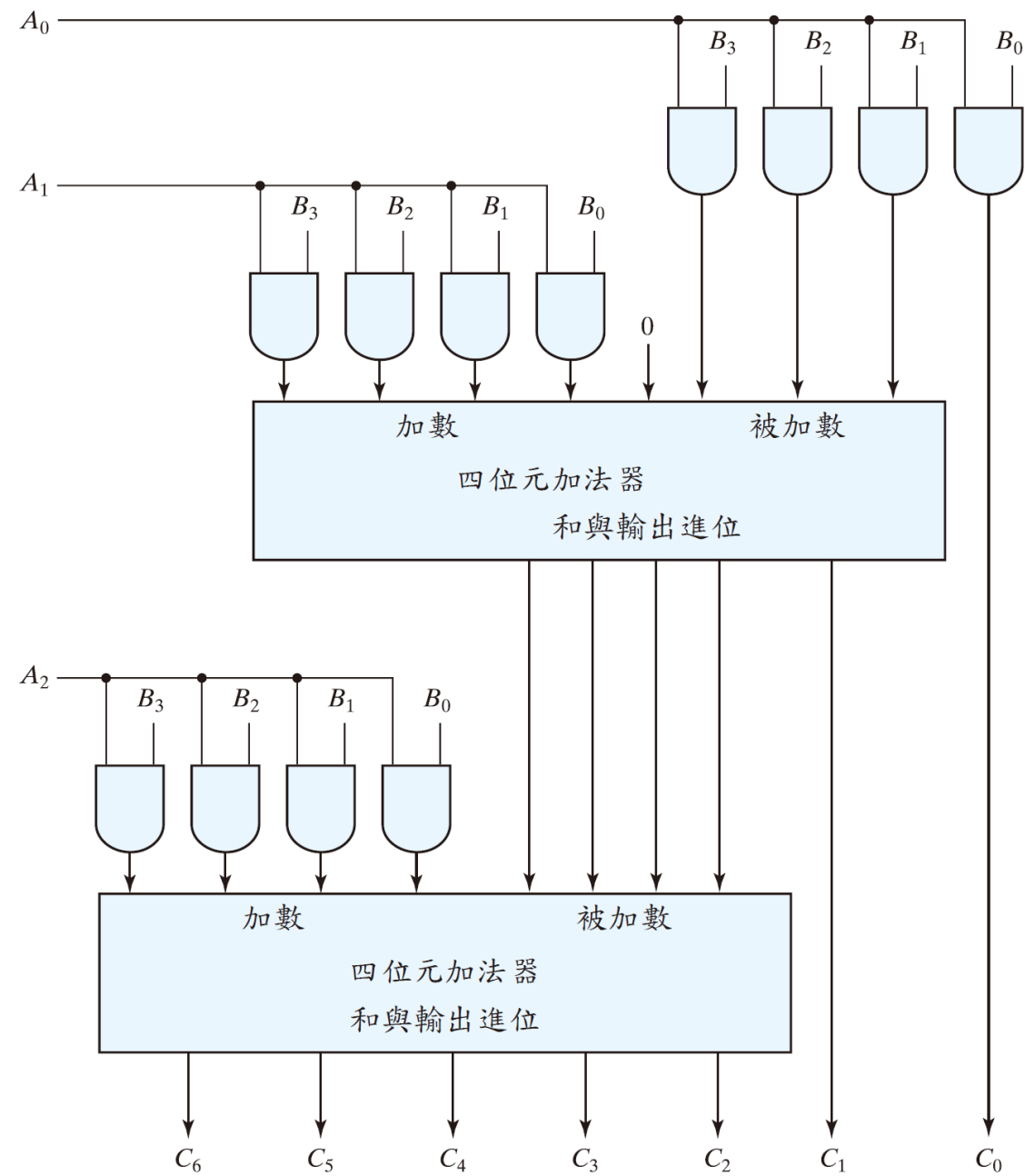


圖 4.16 四位元乘三位元之二進位乘法器

大小比較器

- ▶ **大小比較器(magnitude comparator)** 是用來比較兩個數 A 和 B ，並且決定它們相對大小的組合電路
 - ▶ 輸出： $A > B$ ， $A = B$ 或是 $A < B$
- ▶ 比較兩個數字時，雖然可以用真值表設計，但當位元數增加時會變得太複雜，因此通常利用數字比較的規律來設計比較器。對於兩個 n 位元的比較電路，其真值表中有 2^{2n} 個輸入，即使 $n = 3$ 也會變得很麻煩
- ▶ 因此可以利用此問題本身的規律性。這樣可以：
 - ▶ 降低設計工作量
 - ▶ 減少人為錯誤

4-bit 比較器邏輯閘

- Algorithm -> logic

- $A = A_3A_2A_1A_0$; $B = B_3B_2B_1B_0$
- $A=B$ if $A_3=B_3$, $A_2=B_2$, $A_1=B_1$ and $A_0=B_0$
 - equality: $x_i = A_iB_i + A_i'B_i'$, for $i = 0, 1, 2, 3$
 - $(A=B) = x_3x_2x_1x_0$
- $(A>B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$
- $(A<B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$

- Implementation

- $x_i = (A_iB_i' + A_i'B_i)'$

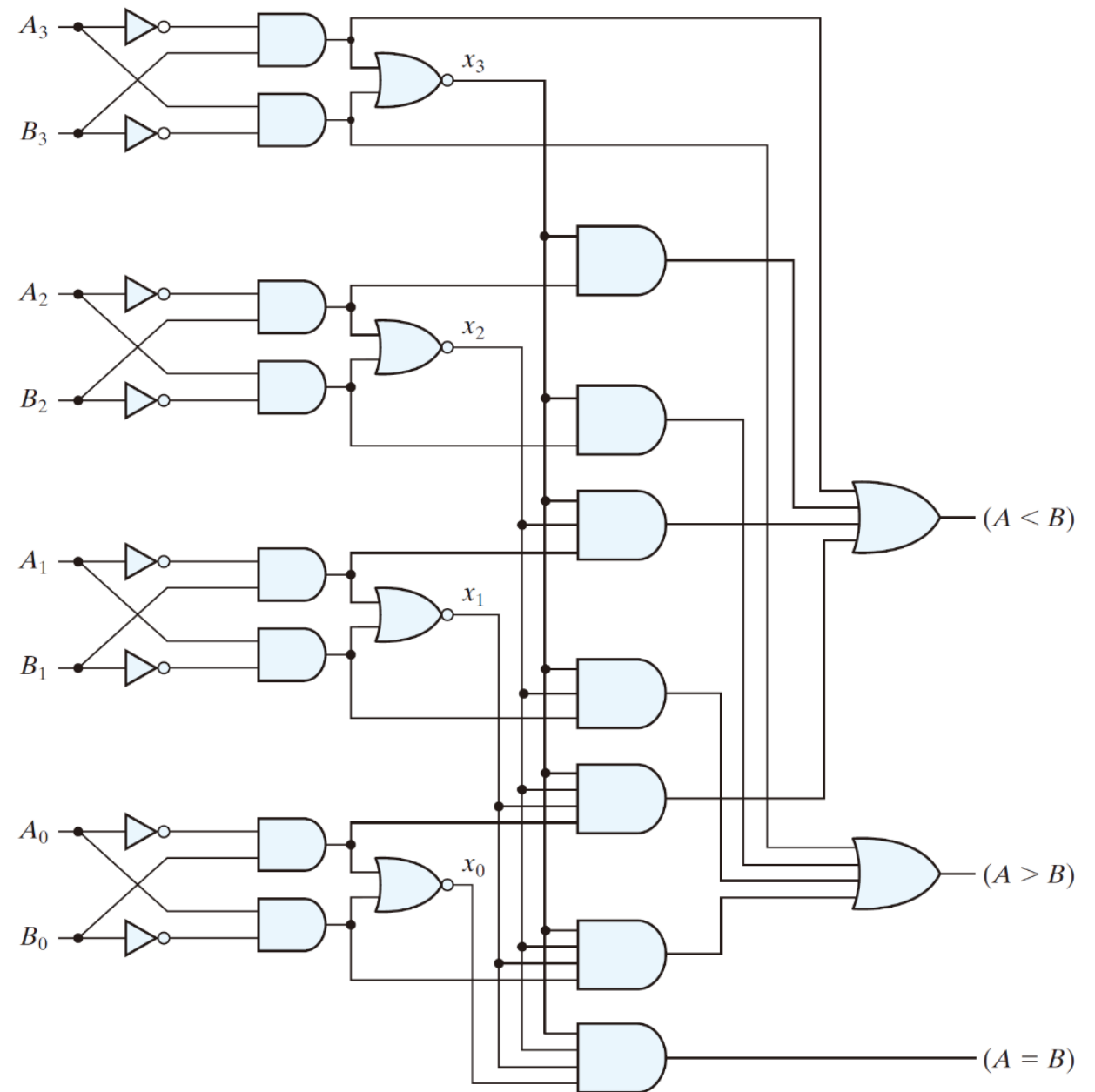


FIGURE 4.17
Four-bit magnitude comparator

練習題 4.7

求出乘積 $(0101)_2 \times (1001)_2$ 。

答案： 0101101_2

解碼器

- ▶ n 到 m 線解碼器(decoder) , 其中 $m \leq 2^n$
 - ▶ 二進位資料從 n 條輸入線 = 2^n 條獨特輸出線
 - ▶ n 位元編碼 , 則解碼器少於 2^n 個的輸出
 - ▶ 對於每一個可能的輸入組合 , 只有一個輸出等於1

Table 4.6
Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

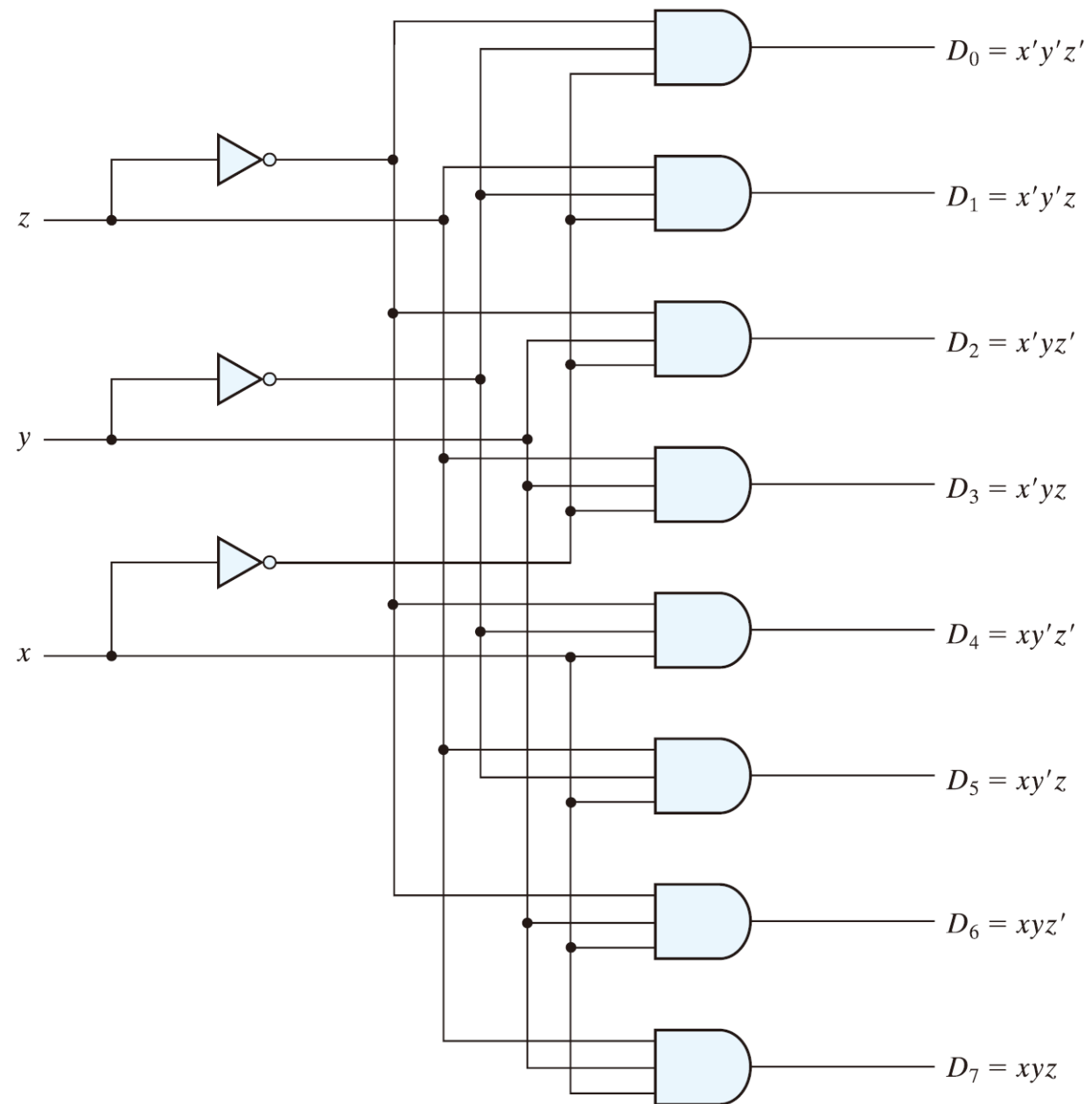


圖 4.18 三對八線解碼器

解碼器

- ▶ 有些解碼器可以用NAND 閘來建立，因為NAND 閘可產生一個具有反相輸出的AND 運算
- ▶ 解碼器還包括一個或更多個**致能(enable)** 輸入來控制電路操作
- ▶ 任何一給定的時間，只有一個輸出等於0，所有其他輸出都等於1

致能 = 「開關」，決定電路 要不要工作

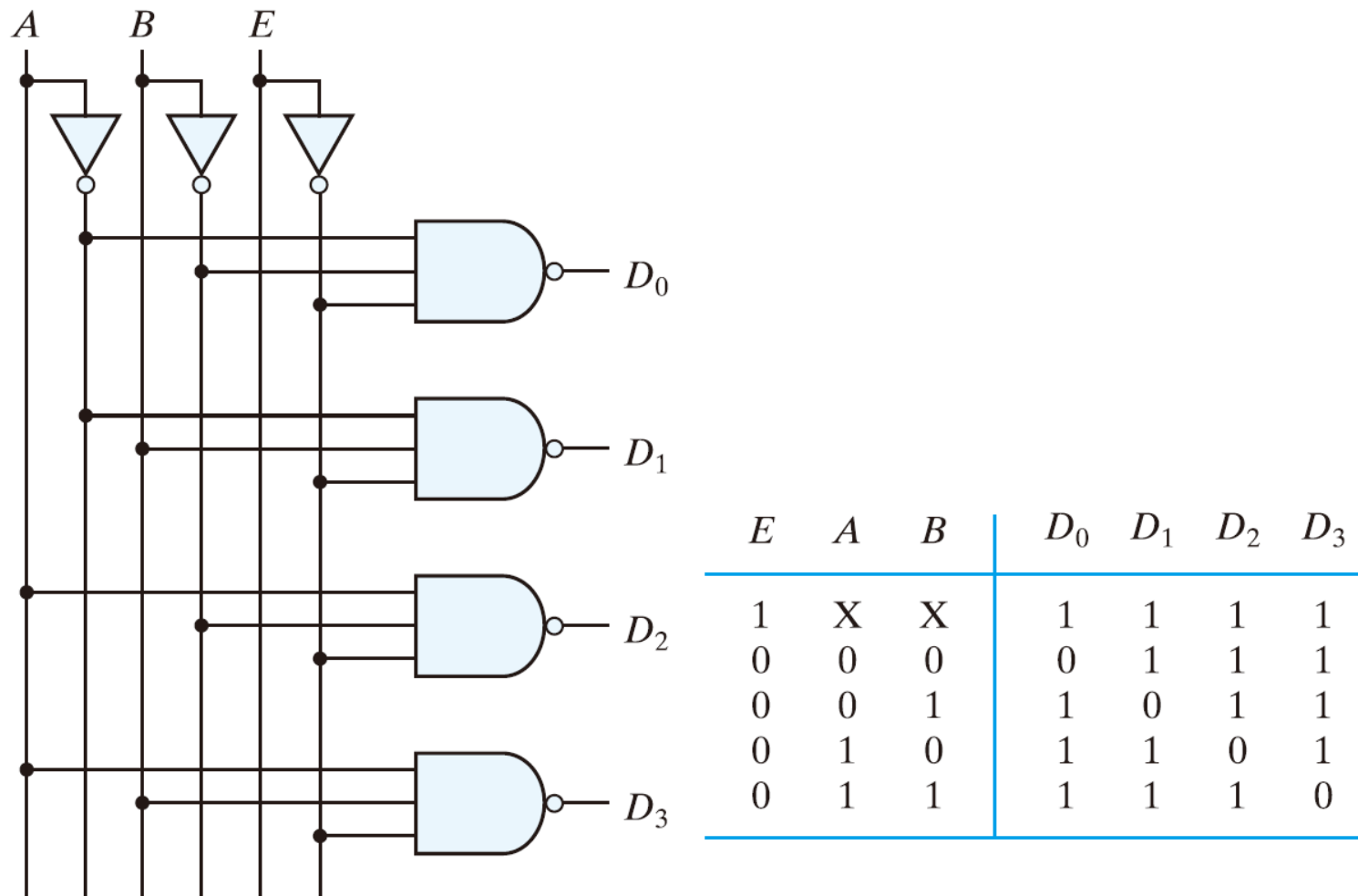


圖 4.19 具有致能輸入的二對四線解碼器

解多工器

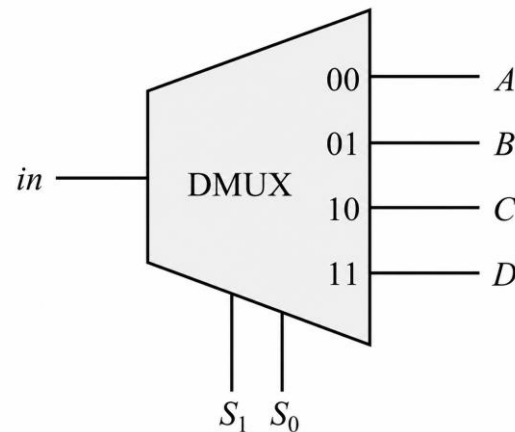
- ▶ 具有致能輸入的解碼器可以被功能化為**解多工器 (demultiplexer)** – 它是一個從單一條線接收資料，並且將它指向 2^n 條可能輸出線之其中一條線的電路
- ▶ 一個具有致能輸入的解碼器稱作**解碼器-解多工器 (decoder-demultiplexer)**

解碼器：是「選哪一條線」

解多工器：是「把資料送到哪一條線」

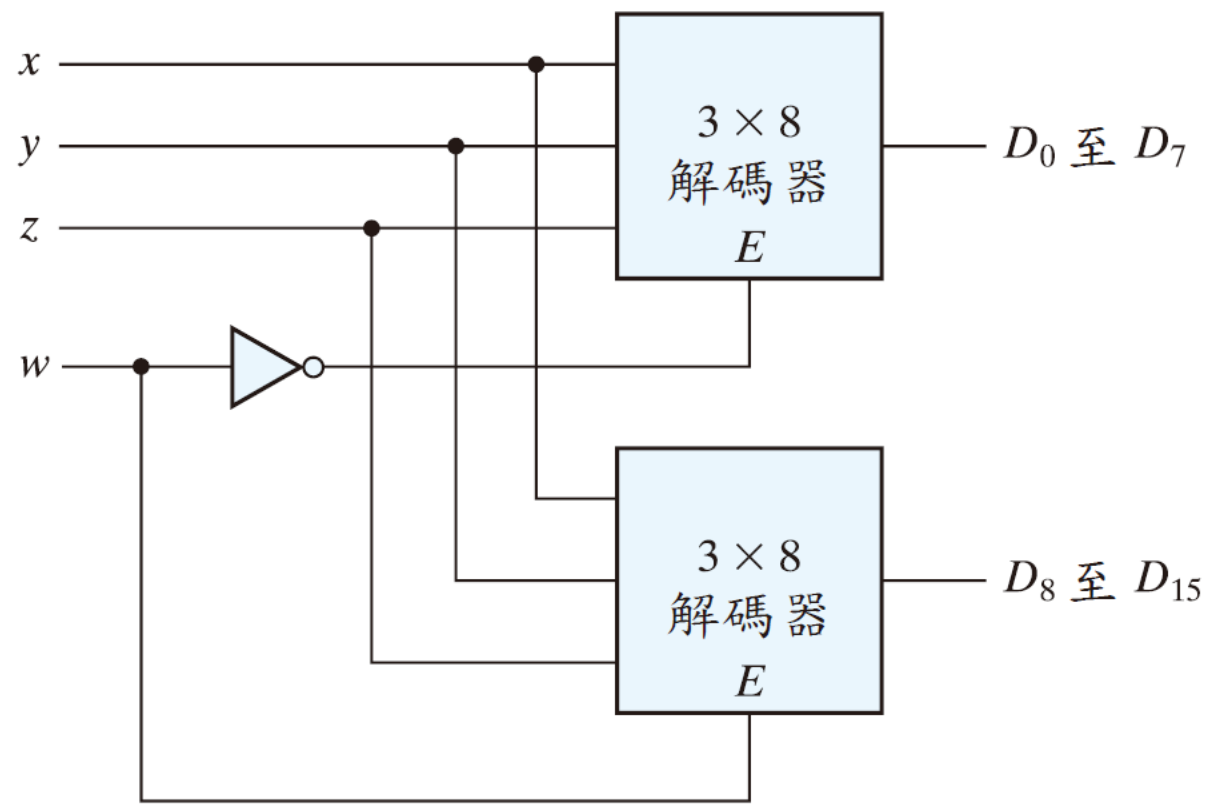
1 對 4 解多工器

- 以 in 表示輸入訊號， S_1 與 S_0 表示選擇訊號線， A 、 B 、 C 、 D 分別表示四條輸出訊號線。1 對 4 解多工器 (1-to-4 demultiplexer) 的功能是根據選擇訊號線 S_1S_0 的組合，決定將輸入訊號 in 傳送到哪一條輸出訊號線。其方塊圖如下所示。
- 1-to-4 解多工器的行為可描述如下：
- 當選擇訊號為 $S_1S_0 = 00$ 時，輸入訊號會傳送至輸出端 A ，即： $A = in$
- 當選擇訊號為 $S_1S_0 = 01$ 時，輸入訊號會傳送至輸出端 B ，即： $B = in$
- 當選擇訊號為 $S_1S_0 = 10$ 時，輸入訊號會傳送至輸出端 C ，即： $C = in$
- 當選擇訊號為 $S_1S_0 = 11$ 時，輸入訊號會傳送至輸出端 D ，即： $D = in$
- 因此，解多工器的重點在於：同一個輸入訊號會依照選擇線的狀態，被導向四個輸出端中的其中一個。



擴展

- 圖4.20所示為具有器



► 圖 4.20 利用兩個 3×8 解碼器建立 4×16 解碼器

組合邏輯電路

- ▶ 任何的布林函數均可以表示成全及項和的形式
- ▶ 任何 n 個輸入和 m 個輸出的組合電路都可以利用一個 n 對線解碼器及 m 個OR 閘來實現
- ▶ 解碼器即可被選擇來產生輸入變數的所有全及項
- ▶ 由全加法器的真值表，得到組合電路以全及項和形式表示的函數：

$$S(x, y, z) = \Sigma(1, 2, 4, 7)$$

$$C(x, y, z) = \Sigma(3, 5, 6, 7)$$

表 4.4 全加法器

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

組合邏輯電路

► 利用解碼器的兩種可能的方法

► OR (F 的全及項) : k 個輸入

► NOR (F' 的全及項) : $2^n - k$ 個輸入

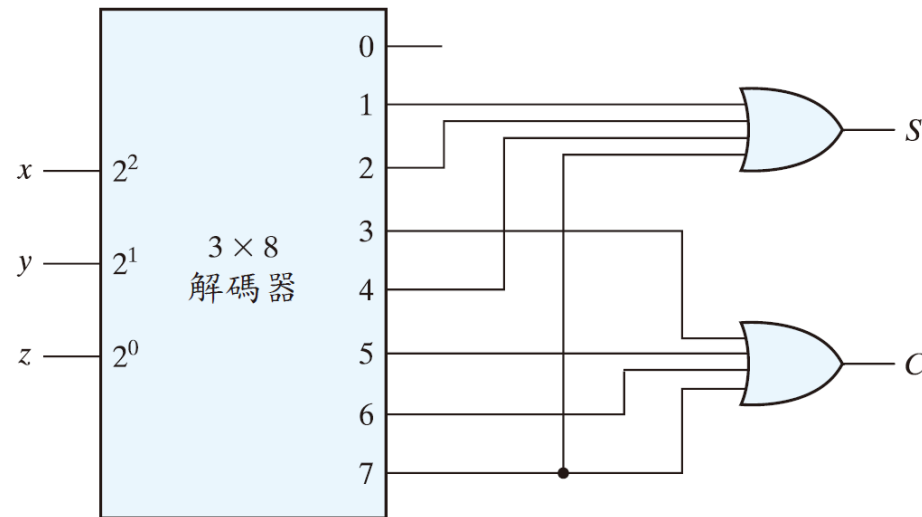


圖 4.21 利用一個解碼器實現全加法器

編碼器

- **編碼器(encoder)** 是一個用來執行解碼器之反向操作的數位電路

Table 4.7
Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$z = D_1 + D_3 + D_5 + D_7$$

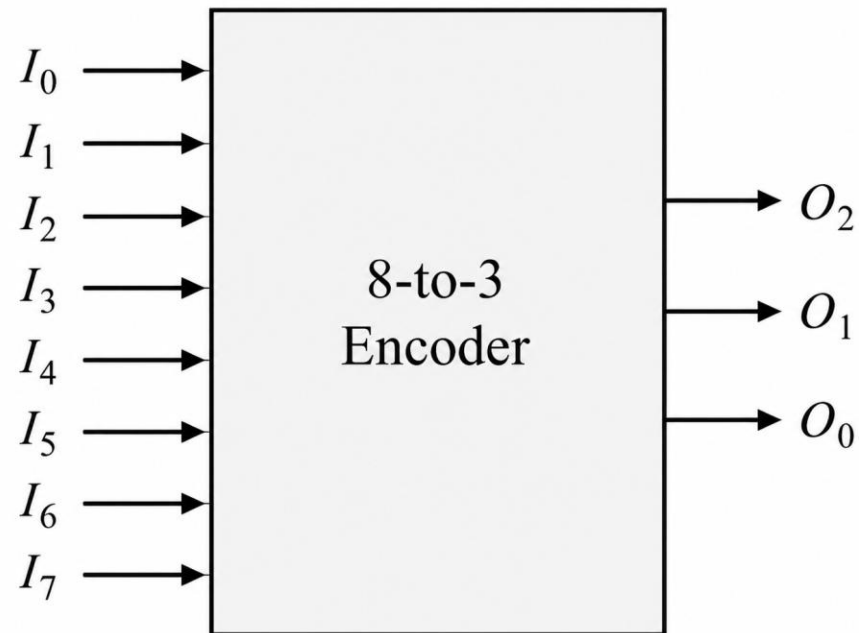
$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

此編碼器可以用三個OR 閘來實現

8 對 3 編碼器

假設使用 $I_0 \sim I_7$ 來分別表示編碼器的八條輸入訊號線，用 O_2 、 O_1 、 O_0 分別表示編碼器的三條輸出訊號線，則此 8 對 3 編碼器 (8-to-3 encoder) 的方塊圖如圖 所示。



優先權編碼器

- 為了解決非法輸入造成的模糊問題，編碼器通常只會對其中一個輸入進行編碼，並使用有效輸出指示器來表示目前輸出是否為有效結果。
- 優先權編碼器的操作是假如有兩個以上的輸入同時等於1，使得具有最高優先權的輸入可以優先

D_3 ：具有最高優先權

D_0 ：具有最低優先權

x ：不理會條件

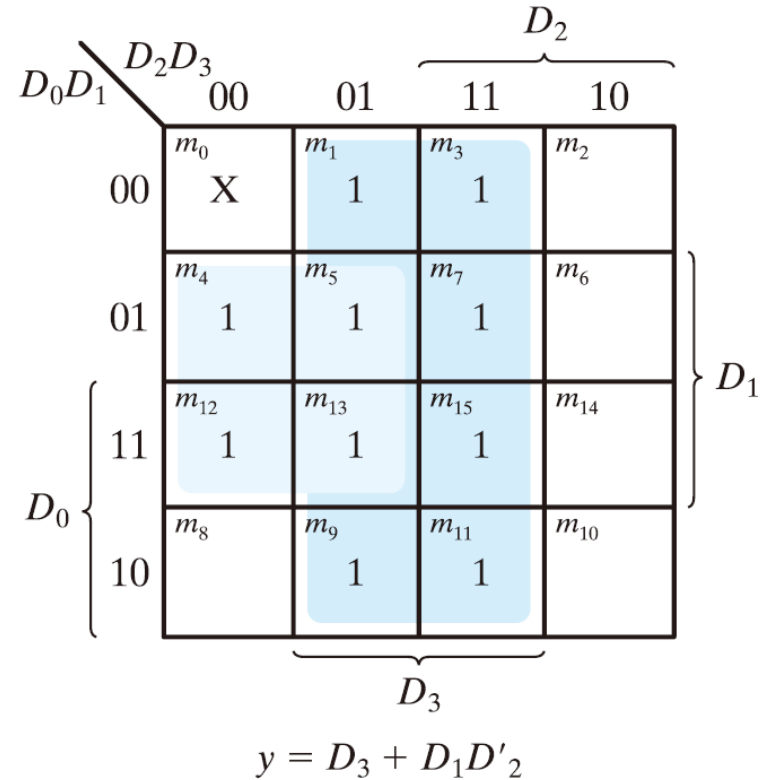
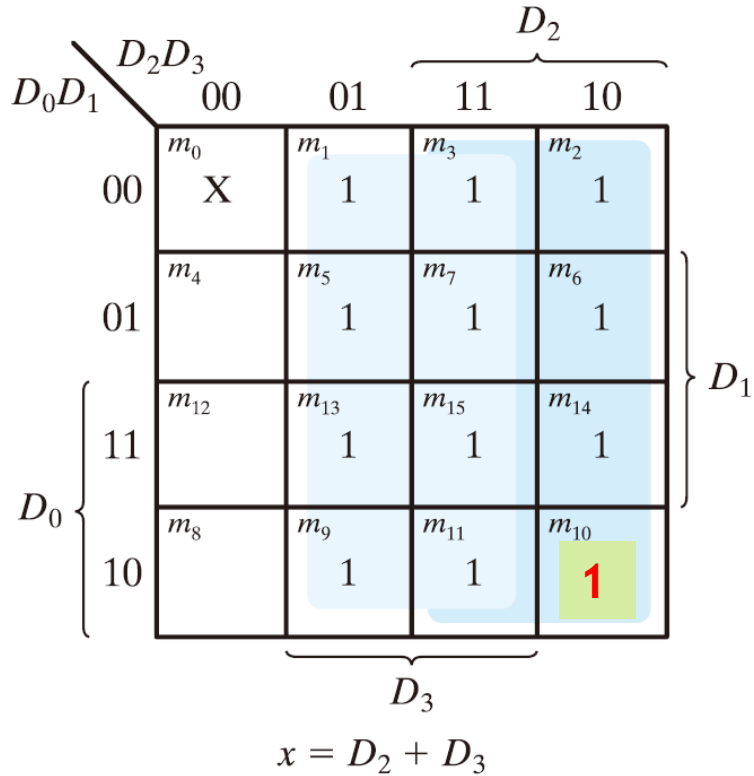
V ：有效(valid) 位元
指示器

Table 4.8
Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

優先權編碼器

► 化簡輸出



► 圖 4.22 優先權編碼器的卡諾圖

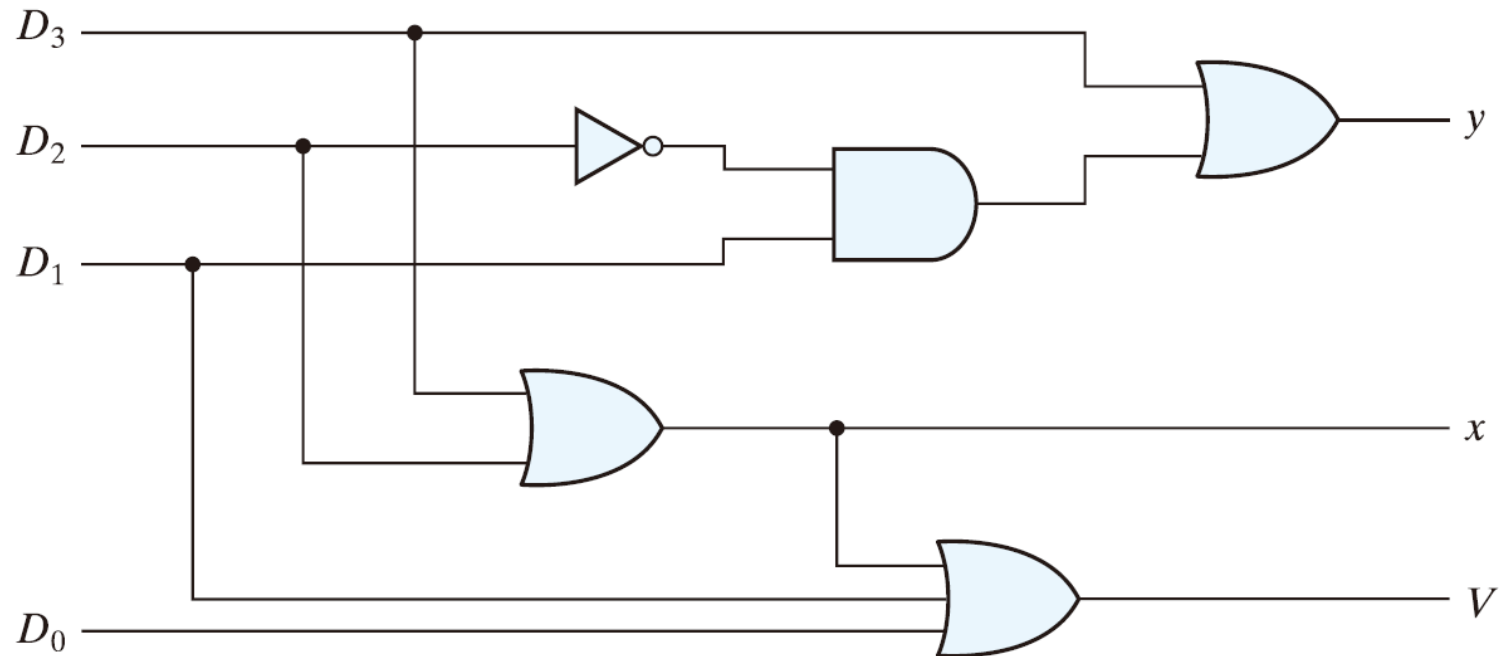
優先權編碼器

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

$$V = D_0 + D_1 + D_2 + D_3$$

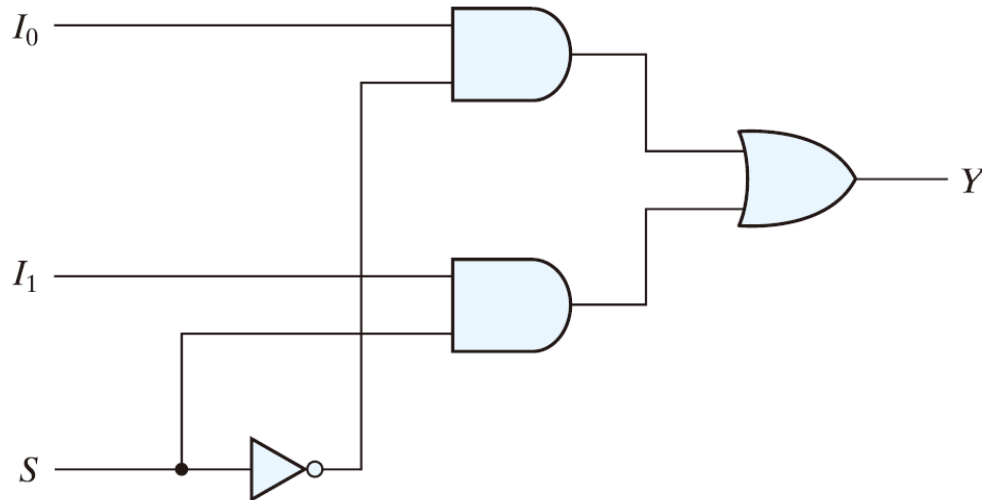
► 圖4.23 實現了優先權編碼器



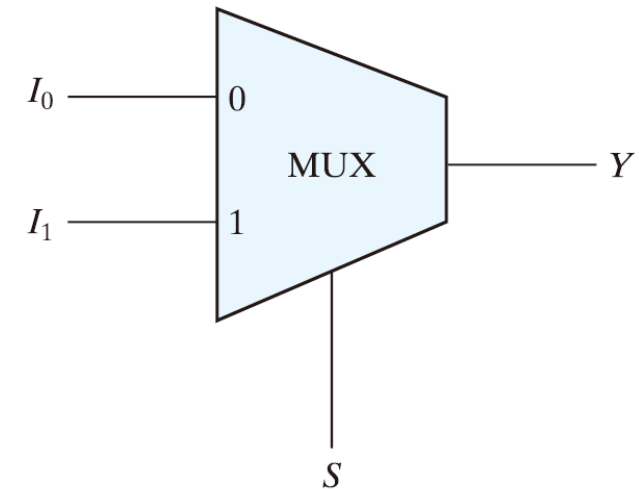
► 圖 4.23 四-輸入優先權編碼器

多工器

- ▶ **多工器(multiplexer)** 是一個由許多條輸入線選擇一條二進位資料並且導向單一輸出線的組合電路
- ▶ 有 2^n 條輸入線、 n 條選擇線，一條輸出線



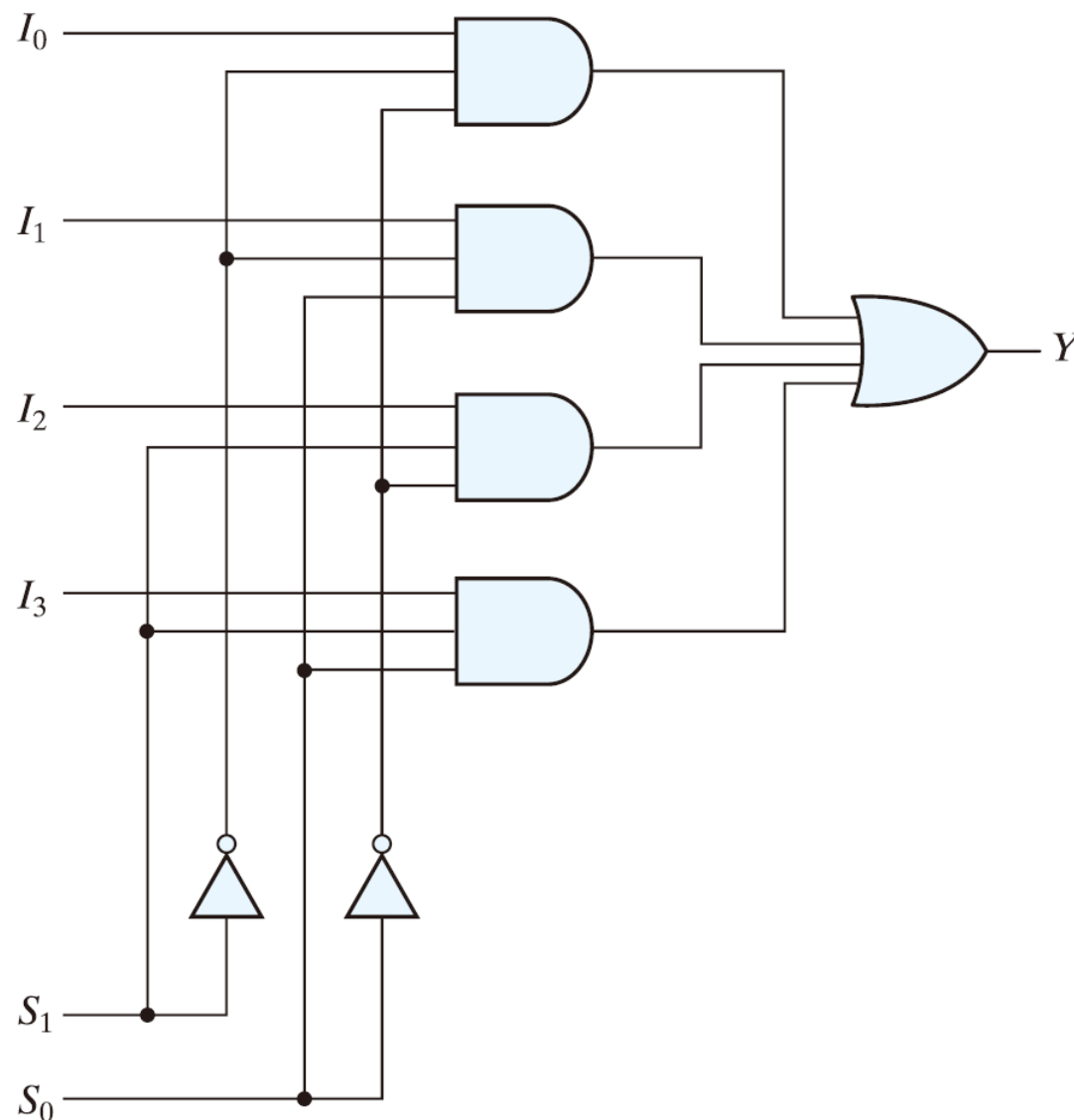
(a) 邏輯圖



(b) 方塊圖

$$Y = I_0S' + I_1S$$

- 2^n 對1 的線多工器:
- n 對 2^n 解碼器
- 將此 2^n 條輸入線中之每一條個別加到每一個AND 閘
- AND 閘的輸出加到單一個OR 閘
- 一條致能輸入(一條選擇線)



(a) 邏輯圖

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) 功能表

圖 4.25 四對一線多工器

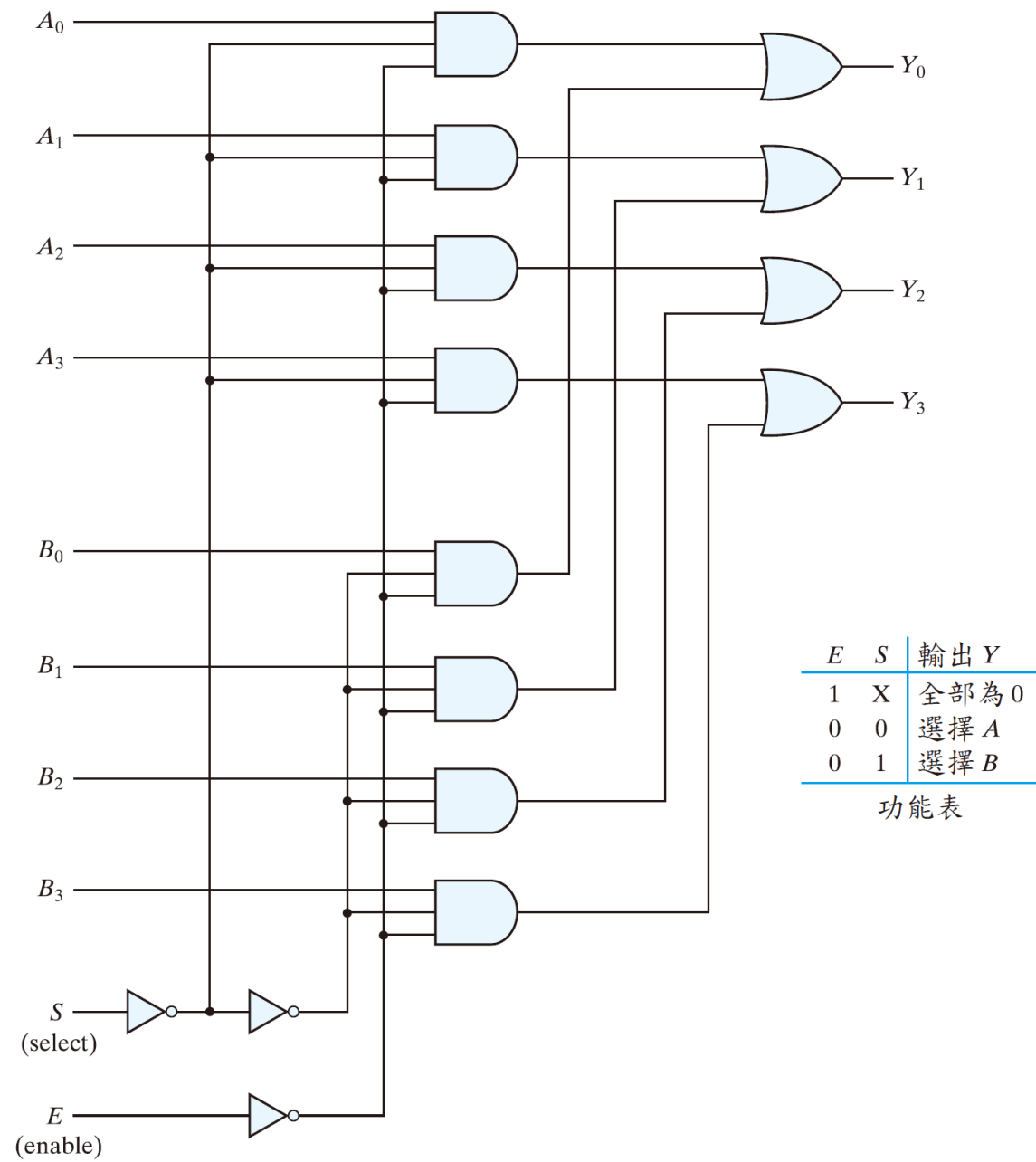


圖 4.26 四重式二對一線多工器

使用多工器實現布林函數

- ▶ 一個多工器的邏輯圖：一個在單元內包含有OR 閘的解碼器
- ▶ 個別的全及項可以由資料輸入來選擇，藉此可提供利用具有 n 條選擇輸入及 2^n 個資料輸入(每一個是一個全及項) 的多工器來實現一個 n 變數布林函數的方法
- ▶ 利用一個具有 $n - 1$ 個選擇輸入之多工器來實現一個 n 變數布林函數較為有效的方法
 - ▶ 函數最前面之 $n - 1$ 個變數連接到多工器的選擇輸入上
 - ▶ 函數所剩下的單一變數做為資料輸入

使用多工器實現布林函數

- ▶ 多工器的邏輯結構可視為一個內部包含 **OR 閘** 的解碼器。多工器會根據選擇輸入的組合，從多個資料輸入中選出其中一路，並將其傳送到輸出端。
- ▶ 由於每一個資料輸入都可以對應到一個全及項（**minterm**），因此可利用具有 n 條選擇輸入與 2^n 個資料輸入的多工器，來實現一個 n 變數的布林函數。其中，選擇輸入用來決定目前要選擇哪一個全及項，而資料輸入則決定該全及項是否會出現在輸出函數中。
- ▶ 不過，較有效率的作法是使用一個具有 $n - 1$ 條選擇輸入的多工器來實現 n 變數布林函數。其基本方法是：將函數中的前 $n - 1$ 個變數接到多工器的選擇輸入端，而剩下的單一變數則作為資料輸入的判斷依據。

使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

固定 select

	<i>x</i>	<i>y</i>	<i>z</i>	<i>F</i>	
<i>I</i> ₀	0	0	0	0	<i>F</i> = <i>z</i>
	0	0	1	1	
<i>I</i> ₁	0	1	0	1	<i>F</i> = <i>z</i> '
	0	1	1	0	
<i>I</i> ₂	1	0	0	0	<i>F</i> = 0
	1	0	1	0	
<i>I</i> ₃	1	1	0	1	<i>F</i> = 1
	1	1	1	1	

(a) 真值表

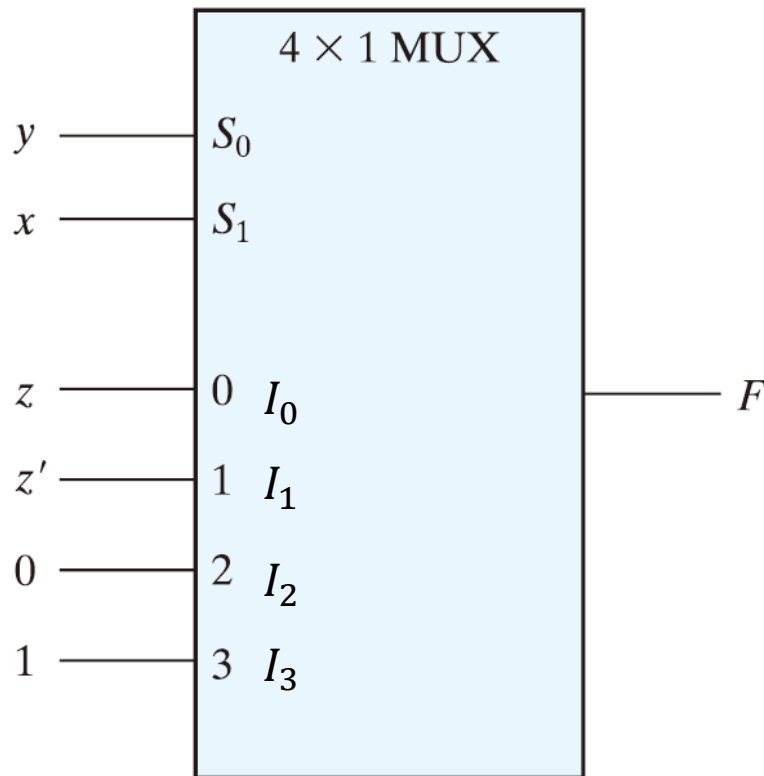
「固定 select，看剩下變數 → 對照 00,11,01,10」

題目是

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

先把它寫成標準和項式：

$$F = x'y'z + x'yz' + xyz' + xyz$$



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

使用多工器實現布林函數

► 步驟

- 首先，將布林函數列在一個真值表中，在表中最前面的 $n - 1$ 個變數應加到多工器的選擇輸入。
- 對於選擇變數的每一個組合，我們求出輸出值做為最後一個變數的函數，這個函數可能是0、1、變數或變數的補數。

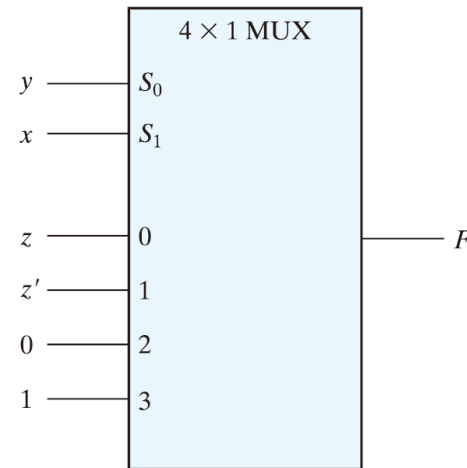
使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

(a) 真值表



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

固定 select (x,y)，看剩下變數 → 對照 00,11,01,10
先把 $F(x, y, z) = \Sigma(1, 2, 6, 7)$ 寫成標準積項和(全及項之和)：
$$F = x'y'z + x'yz' + xyz' + xyz$$

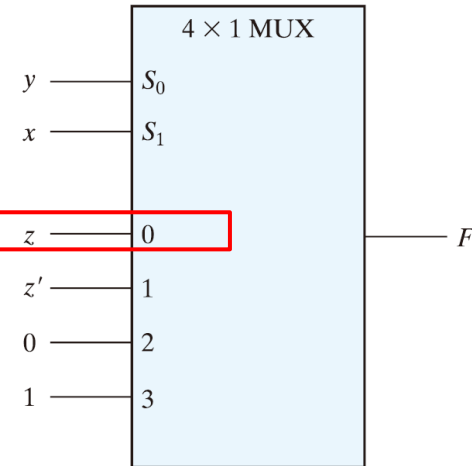
使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) 真值表



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

1. 求 I_0 :

$I_0 = F(0, 0, z)$ ，把 $x = 0, y = 0$ 代入：

$$F(0, 0, z) = 1 \cdot 1 \cdot z + 1 \cdot 0 \cdot z' + 0 \cdot 0 \cdot z' + 0 \cdot 0 \cdot z = z$$

所以 $I_0 = z$

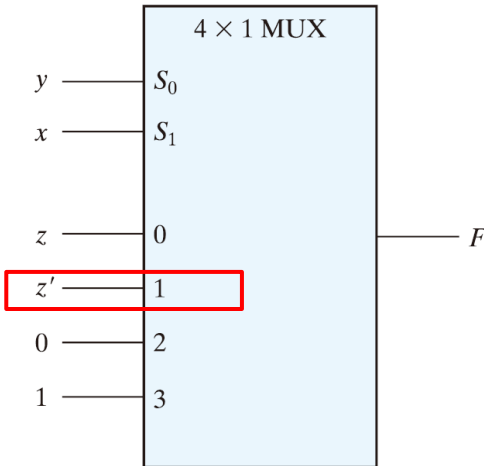
使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

(a) 真值表



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

2. 求 I_1 :

$I_1 = F(0, 1, z)$ ，代入 $x = 0, y = 1$ ：

$$F(0, 1, z) = 1 \cdot 0 \cdot z + 1 \cdot 1 \cdot z' + 0 \cdot 1 \cdot z' + 0 \cdot 1 \cdot z = z'$$

所以 $I_1 = z'$

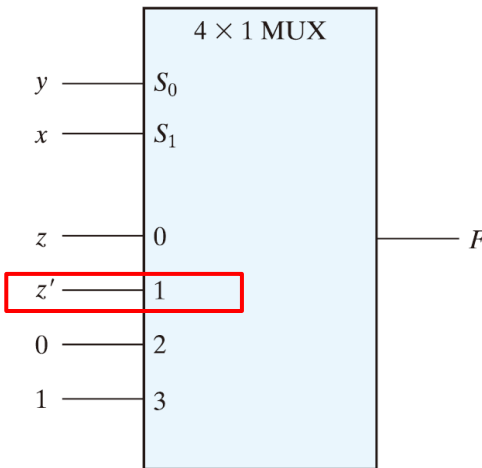
使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

(a) 真值表



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

2. 求 I_1 :

$I_1 = F(0, 1, z)$ ，代入 $x = 0, y = 1$ ：

$$F(0, 1, z) = 1 \cdot 0 \cdot z + 1 \cdot 1 \cdot z' + 0 \cdot 1 \cdot z' + 0 \cdot 1 \cdot z = z'$$

所以 $I_1 = z'$

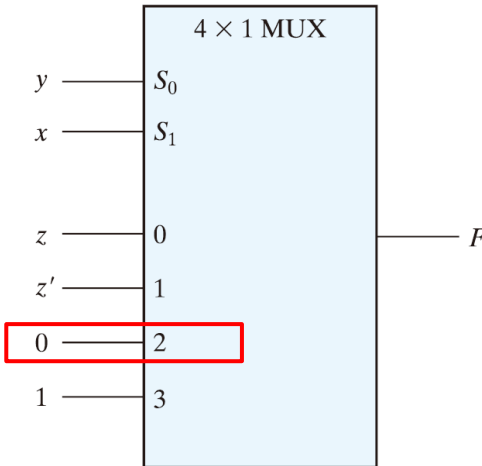
使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) 真值表



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

3. 求 I_2 :

$I_2 = F(1, 0, z)$ ，代入 $x = 1, y = 0$ ：

$$F(1, 0, z) = 0 \cdot 1 \cdot z + 0 \cdot 0 \cdot z' + 1 \cdot 0 \cdot z' + 1 \cdot 0 \cdot z = 0$$

所以 $I_2 = 0$

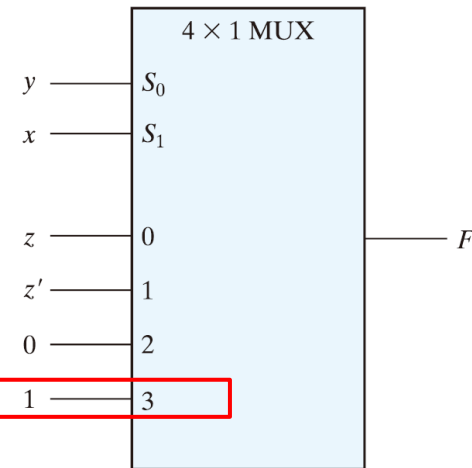
使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) 真值表



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

4. 求 I_3 :

$I_3 = F(1, 1, z)$ ，代入 $x = 1, y = 1$ ：

$$\begin{aligned} F(1, 1, z) &= 0 \cdot 0 \cdot z + 0 \cdot 1 \cdot z' + 1 \cdot 1 \cdot z' + 1 \cdot 1 \cdot z \\ &= z' + z = 1 \end{aligned}$$

所以 $I_3 = 1$

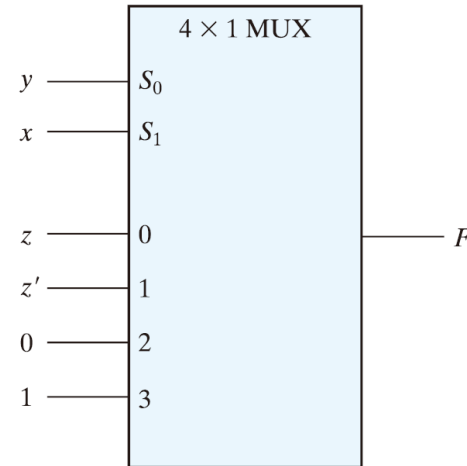
使用多工器實現布林函數

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

► 範例：

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) 真值表



(b) 多工器電路

圖 4.27 利用多工器實現一個布林函數

最後得到

$$I_0 = z, I_1 = z', I_2 = 0, I_3 = 1$$

使用多工器實現布林函數

► 範例：

$$F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

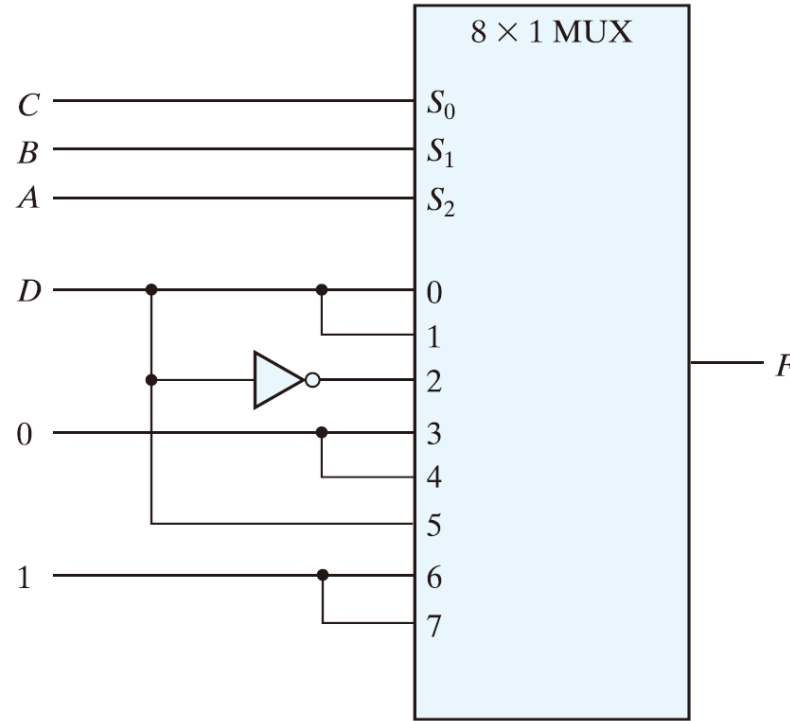


圖 4.28 利用多工器實現一個四-輸入的函數

練習題 4.9

使用多工器來實現布林函數 $F(A, B, C) = \Sigma(3, 5, 6, 7)$ 。

答案：

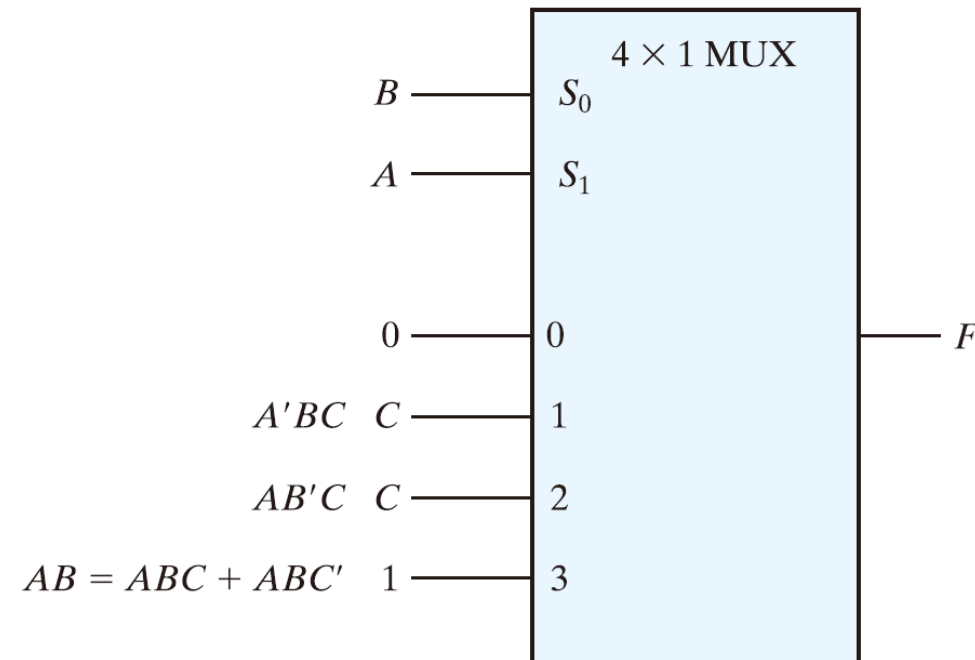


圖 PE4.9

三態閘

- ▶ 多工器可以使用三態閘（ **three-state gates** ）來建構。
- ▶ 三態閘的輸出狀態有三種：0, 1, *高阻抗*
- ▶ 其中，高阻抗狀態也可視為開路狀態。
- ▶ 高阻抗狀態通常稱為：**Tri-state Z**，或簡稱為 **Z 狀態**。

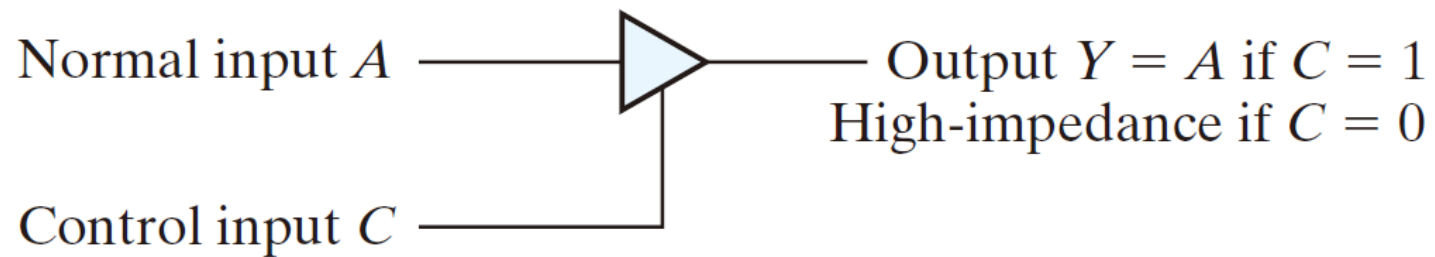
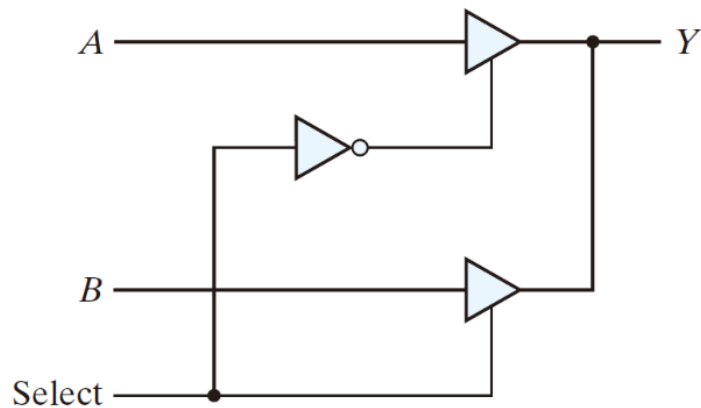


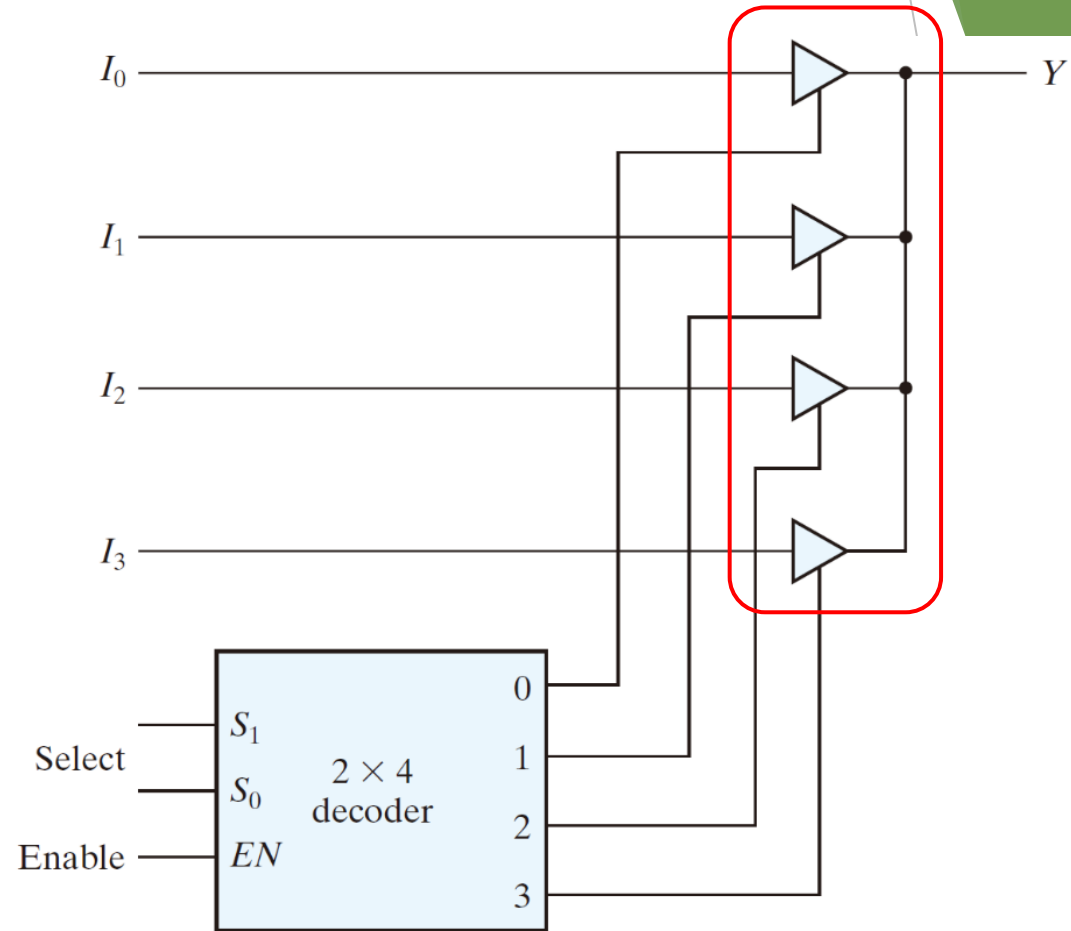
FIGURE 4.29

Graphic symbol for a three-state buffer

四對一線多工器



(a) 2-to-1-line mux



(b) 4-to-1-line mux

FIGURE 4.30
Multiplexers with three-state gates